Análisis de información generada en las plataformas educativas a partir de herramientas de Inteligencia Artificial

Universidad Internacional de Valencia

00 Mes 2026

Titulación:

MIAR – Máster universitario en inteligencia artificial

Curso académico

2025 – 2026

Alumno/a:

Michael Laudrup Luis González

Director/a de TFM: Irma Zoraida Sanabria Cárdenas

De:
Planeta Formación y Universidades

Índice

Resumen 7

Abstract 7

1. Introducción 8

2. Objetivos 11

2.1. Objetivo General 11

2.2. Objetivos Específicos 11

3. Marco teórico y estado del arte 14

3.1. Marco teórico 14

3.1.1. Minería de Datos Educativos (EDM) y Learning Analytics. 14

3.1.2. Heterogeneidad de datos y estándares 15

3.1.3. Deserción frente a bajo rendimiento 17

3.1.4. Aprendizaje No Supervisado: Descubrimiento de Patrones (Clustering) 18

3.1.5. Aprendizaje Supervisado y Modelos de Ensamble 18

3.1.6. Síntesis del marco teórico 20

3.2. Estado del arte. 21

3.2.1. Aprendizaje no supervisado en EDM. 21

3.2.2. Aprendizaje supervisado en EDM. 22

3.2.3. El enfoque híbrido. 24

3.2.4. Inteligencia Artificial Explicable (XAI) en Educación. 25

3.2.5. Síntesis del estado del arte 27

4. Planteamiento del problema y metodología 29

4.1. Planteamiento del problema. 29

4.2. Solución propuesta. 30
4.2.1. Ventanas de tiempo....31
4.2.2. Reducción de dimensionalidad y Aprendizaje no supervisado. 32
4.2.3. Modelado secuencial con Transformer. 33
4.2.4. IA Explicativa. 34
4.2.5. Ecosistema completo 34
4.3. Metodología. 35
4.3.1. Metodología CRISP-DM. 36
4.3.2. Definición de tareas. 38
4.3.3. Planificación. 39
5. Desarrollo del proyecto....41
5.1. Comprensión del negocio. 41
5.2. Análisis exploratorio de datos (EDA). 44
5.3. Preprocesamiento de datos. 49
5.3.1. Preprocesamiento para EDA. 50
5.3.2. Preprocesamiento para Autoencoder y clustering. 51
5.3.3. Preprocesamiento para red neuronal Transformer. 52
5.4. Ingeniería de características....53
5.4.1. Fase I: Características para Autoencoder. 53
5.4.2. Fase II: Características para Transformers....56
5.5. Modelado....58
5.5.1. Autoencoder. 58
5.5.2. Embeddings....61
5.5.3. Clustering....63

5.5.4. Modelado del Transformers. 67

5.5.5. Estrategia del entrenamiento y configuración de clases. 69

5.6. Evaluación 72

5.7. Infraestructura de Ingeniería: Dockerización y Pipelines Automatizados 75

6. Resultados Premilinares 77

7. Conclusiones y trabajos futuros 78

9. Referencias 79

Apéndice I: Repositorio de Código y Reproducibilidad 82

Apendice II: Tablas de características 85

Características del autoencoder. 85

Anexos I: Estructura y diccionario de datos OULAD. 88

Índice de ilustraciones

Ilustración 1 Proceso de preparación del dataset OULAD. Referencia: (Kuzilek, Hlosta, & Zdrahal, 2017) 16

Ilustración 2 Cronograma de tareas definidas. Elaboración propia 40

Ilustración 3 Distribución de estudiantes por resultado académico. Autoría propia - Apéndice 1 - Notebook I 45

Ilustración 4 Análisis comparativo de de interacciones: Ruido diario frente a señal semanal agregada. Autoría propia - Apendice I - Notebook I 46

Ilustración 5 Contraste entre riqueza y resultados académicos - Autoría Profia - Apéndice I - Notebook I 46

Ilustración 6 Comparación de interacciones por curso. Autoría propia - Apéndice I - Notebook I 47

Ilustración 7 Relación entre diferentes métricas y resultados académicos. Apéndice 1 - Notebook I 48

Ilustración 8 Relación entre nivel educativo y resultados académicos Apéndice 1 – Notebook I 49

Índice de tablas

Tabla 1 Descripción de datos de estudiantes 88

Tabla 2 Descripción de entrega de actividades 89

Tabla 3 Descripción datos de interacción de estudiantes 90

Tabla 4 Ficha técnica de repositorio código fuente 82

Resumen

Abstract

1. Introducción

La educación contemporánea se encuentra inmersa en una transformación impulsada por la digitalización de los procesos de enseñanza y aprendizaje. En el centro de este ecosistema digital se sitúan los sistemas de gestión del aprendizaje (LMS, por sus siglas en inglés), plataformas que han evolucionado de ser repositorios estáticos de documentos hasta convertirse en entornos dinámicos y complejos donde se orquesta la experiencia educativa. Entre estos sistemas, Moodle (Modular Object-Oriented Dynamic Learning Environment) se ha consolidado como uno de los LMS más extendidos en el ámbito educativo global, gracias a su arquitectura de código abierto y su flexibilidad pedagógica, siendo adoptado por instituciones para gestionar cursos que van desde la presencialidad hasta la educación a distancia (Gamage, Ayres, & Behrend, 2022). No obstante, a pesar de su uso extendido, el potencial de Moodle para la analítica de aprendizaje sigue estando infrautilizado, lo que hace necesario el diseño de marcos predictivos que aprovechen su analítica interna para identificar dificultades de aprendizaje (Soepriyanto, Nugroho, Nahri, Kesuma, & Setiasih, 2025)

Esta adopción ha traído una generación masiva y continua de datos, conocida como Big Data Educativo. Cada interacción que un estudiante realiza dentro de la plataforma —desde el acceso a un material de lectura, la participación en un foro de discusión, la descarga de una tarea, e incluso la navegación entre menús o la realización de un cuestionario— deja una huella digital registrada en los archivos de registro (*logs*) del sistema. Investigaciones recientes subrayan que estos *logs* por defecto de Moodle son fundamentales para analizar el comportamiento y los hábitos del estudiante, permitiendo modelar desde la retención hasta la carga de trabajo y los estilos de aprendizaje (Soepriyanto, Nugroho, Nahri, Kesuma, & Setiasih, 2025).

Sin embargo, a pesar de esta gran abundancia de datos, la infraestructura tecnológica ha avanzado más rápido que la capacidad pedagógica para explotarla. Existe una brecha significativa entre lo que la tecnología registra y lo que el docente conoce realmente sobre el comportamiento de sus estudiantes. Esta situación genera un escenario donde el LMS se convierte en una "caja negra" de información no procesada, donde la complejidad de las estructuras de datos impide una interpretación directa por parte del profesorado (Khosravi, y otros, 2022).

Los docentes carecen de herramientas para identificar a tiempo qué alumnos están en riesgo de fracaso o qué perfiles de aprendizaje coexisten en su aula, y a menudo detectan las dificultades cuando ya es demasiado tarde para intervenir. Existe, por tanto, una necesidad crítica de desarrollar sistemas inteligentes que actúen como soporte, procesando masivamente estos registros para revelar patrones ocultos y ofreciendo información que permita tomar decisiones pedagógicas informadas y oportunas.

El presente Trabajo de Fin de Máster (TFM) se propone abordar esta problemática mediante la aplicación de técnicas avanzadas de Inteligencia Artificial (IA) y Minería de Datos Educativos (EDM). El propósito principal es transformar los datos brutos generados en plataformas de aprendizaje (LMS) en conocimiento accionable, explorando cómo la modelización computacional puede identificar perfiles de comportamiento y predecir el riesgo académico. De este modo, se busca dotar al docente de herramientas analíticas que superen la mera gestión administrativa y faciliten una toma de decisiones pedagógicas fundamentada y proactiva.

Para dar respuesta a esta problemática y cumplir con el propósito de la investigación, la presente memoria se ha organizado en cinco capítulos que

estructuran el desarrollo del trabajo. A continuación, se describe el contenido de cada uno de ellos:

Tras esta introducción, el Capítulo 2 define los objetivos del proyecto, estableciendo tanto la meta general como los hitos específicos que guían la investigación. Posteriormente, el Capítulo 3 aborda el estado del arte y marco teórico, proporcionando una revisión exhaustiva de la literatura actual y fundamentando los conceptos clave sobre Minería de Datos Educativos y Analítica de Aprendizaje que sustentan la propuesta técnica.

El núcleo del trabajo se desarrolla en el Capítulo 4, titulado desarrollo del proyecto y resultados. Esta sección desglosa la metodología empleada para abordar el estudio y formaliza el planteamiento del problema específico a resolver. A su vez, detalla las fases de desarrollo del proyecto, desde la extracción de los datos hasta la implementación de los algoritmos, culminando con la exposición y análisis de los resultados obtenidos tras la experimentación.

Finalmente, el Capítulo 5 presenta las Conclusiones y trabajos futuros, donde se sintetizan los hallazgos principales, se discute el impacto de la solución en la labor docente y se proponen nuevas líneas de investigación para dar continuidad al estudio. El documento cierra con el apartado de Referencias, que recoge las fuentes bibliográficas y recursos académicos citados a lo largo de la memoria.

2. Objetivos

Tras haber analizado la brecha existente entre la recolección masiva de datos en los entornos virtuales y su limitado aprovechamiento pedagógico, es imperativo formalizar las metas que rigen esta investigación. El propósito de este capítulo es delimitar el alcance del proyecto, estableciendo una hoja de ruta que transite desde la fundamentación teórica hasta la implementación de una arquitectura de *Deep Learning* capaz de transformar registros brutos en conocimiento accionable.

Para garantizar el éxito del sistema predictivo, se han definido metas que equilibran la potencia computacional de los modelos híbridos con la transparencia necesaria para el entorno docente. A continuación, se detalla el fin último del estudio y los hitos operativos que permitirán validar la eficacia y explicabilidad del modelo propuesto.

2.1. Objetivo General

Diseñar, implementar y evaluar una arquitectura híbrida de aprendizaje profundo aplicada a datos de interacción generados en un LMS (dataset OULAD), con el fin de detectar de forma temprana el riesgo de fracaso y abandono académico, integrando técnicas de aprendizaje no supervisado, modelado secuencial y métodos de inteligencia artificial explicable que permitan generar conocimiento pedagógicamente accionable.

2.2. Objetivos Específicos

- Analizar la evolución del estado del arte en minería de datos educativos, contrastando las limitaciones de los métodos lineales clásicos (PCA, Regresión) frente a las capacidades de las arquitecturas profundas

(deep Learning) para modelar la complejidad del comportamiento estudiantil.

- Consolidar y preprocesar el conjunto de datos OULAD, transformando los registros brutos de interacción (logs) en secuencias temporales estructuradas que permitan modelar la evolución de la actividad del estudiante en el LMS y sus patrones de interacción a lo largo del curso, más allá de métricas estáticas acumuladas. Esta representación temporal resulta especialmente adecuada para su tratamiento mediante redes neuronales de tipo transformer y se alinea con propuestas recientes que destacan la utilidad de los "default logs" de Moodle para analizar el comportamiento del estudiante (Soepriyanto, Nugroho, Nahri, Kesuma, & Setiasih, 2025).

- Implementar una arquitectura de Autoencoders (Aprendizaje de Representaciones) para transformar los datos de alta dimensionalidad en un espacio latente comprimido, capaz de capturar relaciones no lineales que escapan a técnicas tradicionales como el PCA.

- Aplicar diferentes técnicas de aprendizaje no supervisado mediante algoritmos de clustering sobre distintas representaciones comprimidas de los datos —espacio latente aprendido por Autoencoder y espacio reducido mediante PCA— con el fin de segmentar la población estudiantil, identificar arquetipos de aprendizaje emergentes y comparar hasta qué punto cada representación preserva o resalta estructura útil.

- Implementar modelos de predicción secuencial basados en Transformers (Aprendizaje Supervisado), aprovechando el mecanismo de Self-Attention para detectar dependencias a largo plazo en el curso.

- Desplegar una estrategia de Explicabilidad Dual (XAI) para mitigar el problema de la "Caja Negra":
    - Intrínseca: Visualización de Mapas de Atención para identificar en qué momentos del curso se focaliza el modelo.
    - Agnóstica: Aplicación de valores SHAP (SHapley Additive exPlanations) —un método basado en la teoría de juegos que permite descomponer la predicción de cualquier modelo para entender el peso de cada factor— con el fin de cuantificar la contribución individual y global de cada variable en la predicción final del éxito o riesgo del estudiante.
- Diseñar un marco de "Explicaciones Accionables", traduciendo los hallazgos técnicos (pesos de atención y valores SHAP) en recomendaciones pedagógicas concretas que permitan al docente realizar intervenciones personalizadas.

3. Marco teórico y estado del arte

La construcción de un sistema predictivo robusto exige un sólido anclaje en la literatura científica y pedagógica actual. Este capítulo establece los cimientos teóricos de la Minería de Datos Educativos (EDM) y la Analítica de Aprendizaje (LA), revisando las investigaciones de los últimos cinco años que sustentan el paso hacia el deep learning.

A través de este análisis, se justifica la adopción de arquitecturas híbridas —basadas en Autoencoders, técnicas de clustering y Transformers— como la solución óptima para capturar la complejidad dinámica del comportamiento estudiantil en entornos virtuales. Asimismo, se posiciona la Inteligencia Artificial Explicable (XAI) como el puente necesario para transformar estos modelos complejos en herramientas pedagógicas transparentes, permitiendo al docente realizar intervenciones fundamentadas y accionables.

3.1. Marco teórico

A continuación, se desarrolla el marco conceptual que define la Minería de Datos Educativos (EDM) y la Analítica de Aprendizaje (LA), para posteriormente analizar el estado del arte de los últimos cinco años. Esta revisión sistemática permite identificar las limitaciones de los métodos tradicionales y justifica la elección de arquitecturas híbridas y técnicas de explicabilidad (XAI) como respuesta a la complejidad del comportamiento estudiantil en plataformas como Moodle.

3.1.1. Minería de Datos Educativos (EDM) y Learning Analytics.

La minería de datos educativos (EDM, Educational Data Mining) y la analítica del aprendizaje (LA, Learning Analytics) son técnicas que utilizan el

análisis de datos para mejorar los procesos educativos. Para ello se extraen patrones y tendencias del rendimiento, el comportamiento y las interacciones de los estudiantes. Estas disciplinas optimizan los procesos de aprendizaje, identifican a los estudiantes con sus dificultades y logran personalizar las experiencias de aprendizaje (Instituto Andaluz Interuniversitario en Ciencia de Datos e Inteligencia Computacional (DASCI), s.f.).

Para materializar estas promesas, es fundamental comprender el origen de la información. Como se detalla en la revisión fundamental sobre EDM publicada en la revista *Modelling*, el primer paso crítico en cualquier ciclo de minería de datos es la recolección de registros provenientes de los "entornos educativos" donde interactúan los estudiantes (Papadogiannis, Wallace, & Karountzou, 2024). Estos entornos, principalmente los Sistemas de Gestión de Aprendizaje (LMS), actúan como la fuente primaria que alimenta los algoritmos.

3.1.2. Heterogeneidad de datos y estándares

En la actualidad existen una gran diversidad de instituciones educativas con plataformas educativas en línea. Cada una de estas plataformas tiene su propio sistema de gestión del aprendizaje, es decir, su propia manera de recolectar datos de los estudiantes, de estructurar la información y de delimitar qué tipo de datos tienen mayor relevancia sobre otros, así como la relación entre estos. Esta falta de estandarización en la captura y estructuración de los datos plantea un desafío metodológico significativo: la dificultad para replicar estudios y generalizar hallazgos entre distintos entornos tecnológicos (Gašević, Dawson, Rogers, & Gasevic, 2016).

A consecuencia de esta situación, se vuelve necesario establecer un marco de referencia común que garantice la validez externa y la comparabilidad de los resultados (benchmarking). En este contexto, el conjunto de datos OULAD

(Open University Learning Analytics Dataset) se ha consolidado como un conjunto de datos de referencia ampliamente utilizado en la investigación sobre Minería de Datos Educativos. Este dataset destaca no solo por su amplia adopción en la literatura científica, sino por su riqueza multidimensional y su riguroso proceso de tratamiento y certificación garantizando la anonimidad de los datos (Véase Ilustración 1)

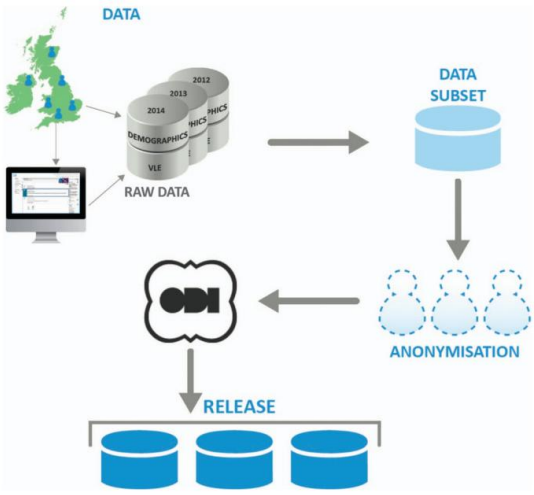

Ilustración 1 Proceso de preparación del dataset OULAD.

Referencia: (Kuzilek, Hlosta, & Zdrahal, 2017)

Integra datos demográficos, resultados de evaluación y registros detallados de interacción (logs) generados en el Entorno Virtual de Aprendizaje (VLE) de la Open University durante los cursos académicos 2013 y 2014. (Kuzilek, Hlosta, & Zdrahal, 2017)

Si bien han surgido conjuntos de datos más recientes en los últimos años, su idoneidad para la presente investigación es limitada. Gran parte de los datasets actuales (procedentes de competiciones tipo Kaggle o plataformas MOOC específicas) presentan inconvenientes críticos: o bien se trata de datos sintéticos generados artificialmente para sortear regulaciones de privacidad —lo que

compromete la fidelidad del comportamiento humano real—, o bien carecen de la documentación académica rigurosa necesaria para sustentar una investigación formal. Asimismo, alternativas masivas como EdNet se centran excesivamente en el trazado de conocimiento (Knowledge Tracing) a nivel de ítem, alejándose de la estructura general de interacción típica de Moodle que este trabajo pretende modelar. Por tanto, OULAD se mantiene como un recurso idóneo para validar modelos predictivos en entornos LMS tradicionales, cuya relevancia continúa siendo reconocida en los inventarios de datos de la literatura científica actual (Papadogiannis, Wallace, & Karountzou, 2024).

3.1.3. Deserción frente a bajo rendimiento

En el ámbito del *Learning Analytics* es crucial diferenciar entre la deserción (dropout) y el bajo rendimiento (failure).

La deserción se define como el cese de la actividad en la plataforma y la desvinculación formal del curso antes de su finalización. Este fenómeno suele estar asociado a factores motivacionales, gestión del tiempo o insatisfacción con el entorno. En el dataset OULAD, esto se representa mediante la etiqueta Withdrawn (retirada). Por el contrario, el bajo rendimiento se refiere a aquellos estudiantes que, completando el ciclo del curso y participando en las evaluaciones, no alcanzan los estándares mínimos de conocimiento, etiquetados como Fail.

Distinguir estos dos fenómenos es fundamental para la modelización computacional, ya que los patrones de interacción que preceden a un abandono (ej. disminución progresiva de accesos) difieren sustancialmente de los patrones de un estudiante que suspende (ej. accesos constantes, pero bajo desempeño en cuestionarios). Un sistema robusto debe ser capaz de diagnosticar ambos riesgos por separado.

3.1.4. Aprendizaje No Supervisado: Descubrimiento de Patrones (Clustering)

El Aprendizaje No Supervisado es una rama de la Inteligencia Artificial donde el modelo trabaja con datos no etiquetados, buscando estructuras ocultas o patrones intrínsecos en la información. En el contexto educativo, su aplicación principal es el clustering o agrupamiento.

A diferencia de la clasificación, donde se le dice al algoritmo "este es un buen alumno", el clustering agrupa a los estudiantes basándose únicamente en la similitud de sus comportamientos (ej. frecuencia de accesos, horarios de conexión, tipos de recursos visitados). Esto permite identificar arquetipos de estudiantes (perfiles) que no son evidentes a simple vista, como "el estudiante intensivo de fin de semana" o "el estudiante que solo lee foros, pero no participa". Algoritmos como K-Means (basado en centroides) o DBSCAN (basado en densidad) permiten segmentar la población estudiantil, proporcionando al docente una taxonomía del aula que facilita la personalización de la enseñanza.

3.1.5. Aprendizaje Supervisado y Modelos de Ensamble

El Aprendizaje Supervisado implica entrenar algoritmos con un conjunto de datos donde se conoce la respuesta correcta (la variable objetivo, en este caso, el resultado final del alumno), para que el modelo aprenda a predecir dicha variable en nuevos estudiantes basándose en su historial. En el contexto de la minería de datos educativos, la evolución de estos modelos ha pasado por tres fases diferenciadas:

- Modelos Lineales: Tradicionalmente, se empleaban técnicas como la Regresión Logística, insuficientes para capturar las relaciones complejas y no lineales del comportamiento estudiantil.
- Modelos de Ensamble (Ensemble Learning): Estudios comparativos recientes sobre el rendimiento estudiantil (Hasan, y otros, 2020) han validado que los algoritmos de ensamble, específicamente Random Forest, superan en precisión a las técnicas clásicas y a las redes neuronales simples cuando se trabaja con datos estructurados. Estos combinan múltiples "árboles de decisión" para manejar el desbalance de clases y determinar la importancia de las variables (Feature Importance). Sin embargo, tal como se evidencia en la metodología empleada por Hasan, y otros (2020), estos modelos dependen de características agregadas (ej. recuento total de visualizaciones), lo que reduce la compleja dinámica temporal del aprendizaje a valores estáticos acumulados, perdiendo el contexto secuencial.
- Deep learning Secuencial y Transformers: Dado que el aprendizaje es un proceso dinámico que evoluciona en el tiempo, el estado del arte más avanzado se ha desplazado hacia redes neuronales capaces de procesar secuencias (Kusumawardani & Alfarozi, 2023).

Inicialmente, las redes recurrentes (RNN y LSTM) permitieron analizar la evolución temporal del alumno. Más recientemente, la arquitectura transformer ha ganado protagonismo en el modelado secuencial. A diferencia de las anteriores, los transformers se basan en mecanismos de autoatención (self-attention), que permiten ponderar la relevancia de cada evento del curso en relación con los demás, independientemente de la distancia temporal entre ellos. De este modo, es posible capturar dependencias a largo plazo (por ejemplo, cómo el desempeño en una tarea temprana puede influir en el abandono en semanas posteriores) (Kusumawardani & Alfarozi, 2023; Kuzilek, Hlosta, & Zdrahal, 2017).

3.1.6. Síntesis del marco teórico

En síntesis, el marco teórico expuesto evidencia que tanto las técnicas no supervisadas como las supervisadas poseen fortalezas complementarias. Mientras que el clustering permite descubrir la estructura subyacente de los datos sin sesgos previos, los modelos supervisados capitalizan esa información para realizar predicciones concretas. Esta complementariedad teórica sugiere que la integración de ambas ramas —en un enfoque híbrido— constituye la arquitectura idónea para abordar la complejidad del comportamiento estudiantil en plataformas como Moodle, premisa que guiará la revisión de la literatura y la propuesta metodológica de este trabajo.

3.2. Estado del arte.

La literatura científica reciente en minería de datos educativos ha evolucionado desde la aplicación aislada de algoritmos básicos hacia arquitecturas cada vez más complejas orientadas a la personalización. Para contextualizar la contribución de este TFM, esta sección analiza las investigaciones más relevantes de los últimos cinco años que han utilizado el dataset OULAD. El análisis se estructura focalizándose en la evolución metodológica: partiendo de los enfoques puramente exploratorios (no supervisados) y predictivos (supervisados), hasta llegar a las propuestas híbridas más vanguardistas. Asimismo, se identifican las limitaciones recurrentes en los estudios actuales —específicamente en la reducción de dimensionalidad y la interpretabilidad— que justifican la necesidad de explorar técnicas no lineales de Deep Learning.

3.2.1. Aprendizaje no supervisado en EDM.

En el contexto específico del dataset OULAD, la investigación más reciente no solo valida la eficacia de las técnicas de clustering, sino que las posiciona como herramientas indispensables para la personalización educativa. Un ejemplo es el estudio publicado por El Ghali, Atouf, El Guemmat, Broumi, & Talea (2025), quienes profundizaron en la segmentación estratégica de estudiantes utilizando este conjunto de datos. Su investigación contrastó algoritmos basados en vecindad (KNN) frente al clustering jerárquico, empleando PCA (Análisis de Componentes Principales) para gestionar la alta dimensionalidad de las variables demográficas y de interacción.

Los resultados presentados por El Ghali et al. marcan el estándar del estado del arte actual: la aplicación del PCA no solo redujo el tiempo de cómputo en un 60%, sino que, combinado con clustering jerárquico, permitió alcanzar métricas de calidad excepcionales. Mediante esta metodología, lograron identificar cuatro

arquetipos de estudiantes, revelando perfiles complejos como aquellos con "alta participación, pero bajo rendimiento" o patrones de compromiso erráticos (El Ghali, Atouf, El Guemmat, Broumi, & Talea, 2025).

Estos hallazgos son relevantes para la presente investigación, ya que sugieren que el dataset OULAD contiene estructura y regularidades aprovechables. No obstante, el hecho de que el estudio reciente se apoye metodológicamente en PCA también refleja que el campo tiende a priorizar transformaciones lineales por su simplicidad e interpretabilidad. En este contexto, se propone explorar el uso de técnicas no lineales (autoencoders) como una vía complementaria para evaluar si pueden representar aspectos del comportamiento estudiantil que PCA no capture de forma directa. Esta propuesta se plantea de manera exploratoria, sin asumir a priori una mejora garantizada, y su utilidad se determinará empíricamente a partir de comparativas y métricas en las fases posteriores del trabajo.

3.2.2. Aprendizaje supervisado en EDM.

La evolución de los modelos predictivos aplicados a OULAD muestra una transición desde clasificadores lineales hacia métodos de ensamble y, más recientemente, hacia enfoques secuenciales de aprendizaje profundo. Este desplazamiento es coherente con la propia naturaleza del dataset OULAD, que combina variables demográficas, evaluaciones y trazas de interacción en el VLE para más de 32.000 estudiantes, y cuyo uso extendido facilita la comparación metodológica entre estudios (Kuzilek, Hlosta, & Zdrahal, 2017)

En tareas con variables tabulares agregadas, algoritmos de gradient boosting como XGBoost se emplean con frecuencia como línea base competitiva; por ejemplo, Firat (2025) reporta mejores resultados de XGBoost frente a Random Forest en su configuración experimental, destacando además la

dependencia de estos modelos respecto a la ingeniería manual de características. En este sentido, parte de la literatura enfatiza que la ganancia de rendimiento en ensambles se apoya en "cómo" se agregan los logs (por semana/día, por tipo de actividad, por curso) y en la selección de variables, lo que puede limitar la captura de patrones temporales finos cuando se reduce la secuencia a totales acumulados.

Para incorporar la dimensión temporal de los registros del LMS, trabajos recientes han evaluado modelos secuenciales: Torkhani & Rezgui (2025) obtienen alrededor de un 83% de accuracy con LSTM, superando a varios enfoques clásicos en su comparación. Este resultado es consistente con la línea de trabajos que tratan los clics/interacciones como series temporales (p. ej., agregación semanal) para anticipar riesgo académico antes de finalizar el curso, priorizando el "cuándo" además del "cuánto" interactúa el estudiante. Además, otros enfoques recientes amplían el foco a escenarios multiclase (pass, fail, withdrawn, distinction) y muestran que redes neuronales pueden mejorar la predicción temprana cuando integran diversas fuentes (demografía, evaluaciones y clickstream) con preprocesado y feature engineering más exhaustivos (Junejo, y otros, 2024).

En esta misma línea, estudios como Kusumawardani & Alfarozi (2023) exploran transformer encoders sobre OULAD para predicción secuencial (diaria/semanal), reportando resultados competitivos y ventajas prácticas asociadas a la atención y la paralelización del entrenamiento. De forma complementaria, investigaciones recientes sobre OULAD centradas específicamente en abandono (withdrawal) también han propuesto variantes transformer y reportan un rendimiento alto y estable en predicción temprana cuando se usan secuencias semanales de actividad (Luo, Yu, Xie, Wu, & Xu, 2024). Este tipo de evidencias refuerza la idea de que, cuando se preserva la estructura secuencial de los logs, la autoatención puede competir (y en algunos

diseños superar) a enfoques recurrentes en tareas de detección temprana de riesgo.

Más allá de la eficiencia, una posible ventaja de los transformers para este TFM es su utilidad interpretativa como señal complementaria: los mecanismos de autoatención generan pesos que permiten analizar qué partes de la secuencia (por ejemplo, interacciones en semanas concretas) han sido más relevantes para la predicción. No obstante, la literatura suele recomendar tratar la atención como una señal interpretativa "auxiliar" y complementarla con métodos de explicabilidad agnósticos al modelo para obtener justificaciones más robustas. En consecuencia, este TFM propone no solo evaluar modelos transformer en términos de rendimiento, sino también analizar patrones de atención y complementarlos con técnicas agnósticas como SHAP para reforzar la explicabilidad (XAI) del sistema híbrido, facilitando que el docente comprenda el "porqué" del riesgo detectado.

3.2.3. El enfoque híbrido.

Los modelos híbridos representan la vanguardia en la minería de datos educativos, combinando técnicas no supervisadas y supervisadas en una arquitectura secuencial. A diferencia de los modelos monolíticos tradicionales, el enfoque híbrido segmenta primero la población mediante clustering para luego entrenar predictores especializados en cada perfil.

Esta estrategia ha sido validada empíricamente por Al-Tameemi et al. (2024) quienes demostraron que el uso de clústeres previos mejora significativamente la precisión de la clasificación en entornos educativos. No obstante, existe un patrón recurrente en la literatura de vanguardia, observado tanto en Al-Tameemi et al. como en El Ghali et al. (2025): la omnipresencia del PCA como técnica estándar de reducción de

dimensionalidad, lo que implica una presuposición de que la relación entre los datos es lineal.

Dado que el aprendizaje humano es un proceso complejo y dinámico, la asunción de linealidad puede resultar limitante en ciertos contextos y simplificar relaciones potencialmente más ricas en los datos. Por ello, este TFM plantea complementar el enfoque basado en PCA con una exploración de arquitecturas de autoencoders (Deep Learning) como alternativa no lineal de reducción de dimensionalidad. El objetivo es evaluar empíricamente si estas representaciones latentes son capaces de capturar dependencias que PCA no modela de forma directa y, en caso de aportar valor, utilizarlas como base para los modelos predictivos posteriores.

3.2.4. Inteligencia Artificial Explicable (XAI) en Educación.

Tal como se adelantó al finalizar el análisis sobre los modelos supervisados y la arquitectura Transformer, la búsqueda de una mayor precisión predictiva conlleva la creación de "cajas negras", modelos con estructuras complejas que no son fácilmente interpretables (Khosravi, y otros, 2022).En el ámbito educativo, un modelo que predice el fracaso con un 90% de acierto pero que no explica las causas es pedagógicamente estéril, ya que impide al docente diseñar una intervención correctiva fundamentada. La literatura define esto como la necesidad de generar "explicaciones accionables" (actionable explanations), entendidas como datos que permiten establecer un procedimiento correctivo o bucle de retroalimentación para un conjunto de acciones (Khosravi, y otros, 2022).

Estudios recientes coinciden en que la adopción real de la Minería de Datos Educativos depende de la confianza (Trustworthiness) que el usuario final deposite en el sistema, siendo la explicabilidad el vehículo fundamental para

incrementarla. Los modelos de Deep Learning avanzados, como las redes LSTM o los propios Autoencoders, operan mediante transformaciones no lineales complejas que oscurecen la relación directa entre la entrada y la salida; De hecho, la estructura de estas redes profundas dificulta su comprensión teórica incluso para expertos en la materia, debido a la geometría de sus espacios de alta dimensión (Sejnowski, 2020). Esta opacidad genera una barrera crítica: el docente no puede distinguir si una predicción de riesgo se debe a una falta de actividad, a un bajo rendimiento o a un patrón de comportamiento anómalo.

Para mitigar esta problemática sin renunciar a la potencia de los modelos no lineales, este TFM adopta una estrategia de explicabilidad dual, alineada con el marco de trabajo XAI-ED:

- Explicabilidad Intrínseca (Attention Maps): Aprovechando la arquitectura Transformer seleccionada, se extraerán y visualizarán las matrices de pesos de atención (Self-Attention Weights). A diferencia de las redes recurrentes, estos mapas permiten observar directamente en qué momentos del curso o actividades específicas se "fijó" el modelo para determinar el riesgo de un estudiante Este enfoque sigue las recomendaciones de complementar modelos complejos con componentes visuales interpretables para garantizar la transparencia (Khosravi, y otros, 2022).
- Explicabilidad Agnóstica del Modelo (SHAP): Dado que la arquitectura propuesta es híbrida, la interacción entre componentes puede ser compleja. Para garantizar una interpretación global, se utilizarán valores SHAP (Shapley Additive exPlanations). Esta técnica, fundamentada en la teoría de juegos, permite asignar una puntuación de contribución a cada variable, desvelando qué características empujaron la predicción hacia el éxito o el fracaso y distribuyendo equitativamente la importancia entre ellas (Lundberg & Lee, 2017).

3.2.5. Síntesis del estado del arte

La revisión bibliográfica realizada, si bien no pretende abarcar la totalidad de la vasta producción científica en Minería de Datos Educativos, ha permitido identificar ciertas tendencias y patrones predominantes en los estudios recientes (2020-2025) sobre el dataset OULAD. A partir de la muestra seleccionada, se extraen tres conclusiones que fundamentan la propuesta experimental de este TFM:

1. Exploración de la no-linealidad: En los trabajos analizados (como El Ghali et al., 2025), se observa un uso frecuente de técnicas lineales como el PCA para la reducción de dimensionalidad. Si bien estas técnicas son efectivas, cabe la posibilidad de que la complejidad del comportamiento estudiantil contenga matices no lineales que se pierden en estas proyecciones. Por ello, resulta pertinente explorar si arquitecturas de Deep Learning, como los Autoencoders, pueden generar representaciones latentes más ricas y mejorar el rendimiento de los modelos posteriores en este contexto específico.

2. Evolución hacia modelos secuenciales: Aunque las redes LSTM han mostrado buenos resultados en la literatura consultada, los estudios más recientes comienzan a señalar las arquitecturas basadas en atención (Transformers) como una alternativa prometedora para capturar dependencias a largo plazo. Este trabajo busca contribuir a esta línea de investigación emergente, aplicando y validando la eficacia de los Transformers sobre los datos de la Open University.

3. La integración de la explicabilidad (XAI): Coincidiendo con la visión de autores como Sejnowski (2020) y Khosravi et al. (2022), se detecta una necesidad transversal de dotar de transparencia a los modelos

complejos. Más allá de la precisión predictiva, este TFM asume la premisa de que la adopción real de estas herramientas depende de su interpretabilidad. Por tanto, se propone evaluar una estrategia de explicabilidad dual (SHAP y Attention Maps) para determinar si ofrece información pedagógicamente accionable al docente.

En definitiva, basándonos en el alcance de esta revisión, se constata que, si bien estas tecnologías han sido exploradas de manera aislada o parcial, no se ha hallado evidencia en la literatura reciente de una arquitectura que las orqueste simultáneamente sobre el dataset OULAD.

Por consiguiente, la combinación específica de Autoencoders (para la representación latente no lineal), Transformers (para el modelado secuencial) y XAI (para la explicabilidad dual) se presenta no solo como una arquitectura híbrida de alto interés experimental, sino como una propuesta novedosa que busca cubrir el vacío existente. Este enfoque pretende verificar si la integración sinérgica de estas tres técnicas avanzadas ofrece ventajas tangibles y superiores frente a la aplicación fragmentada o tradicional observada en el estado del arte.

4. Planteamiento del problema y metodología.

Este capítulo presenta el desarrollo técnico del sistema propuesto para la detección temprana del riesgo académico a partir de los datos del dataset OULAD. Tras la fundamentación teórica expuesta en los capítulos anteriores, se formaliza aquí el problema desde una perspectiva computacional y se describe la metodología seguida para su resolución.

En primer lugar, se delimita el planteamiento del problema, identificando las limitaciones actuales en el aprovechamiento pedagógico de los registros generados en los LMS. A continuación, se detalla la solución propuesta, basada en una arquitectura híbrida que combina aprendizaje no supervisado y modelado secuencial.

Finalmente, se expone el marco metodológico adoptado —estructurado bajo el estándar CRISP-DM— así como las fases operativas que guían el preprocesamiento, el entrenamiento de los modelos y la evaluación del sistema, con especial atención a su interpretabilidad y aplicabilidad en el contexto docente.

4.1. Planteamiento del problema.

En la actualidad, la digitalización educativa ha convertido a los Sistemas de Gestión del Aprendizaje (LMS), especialmente Moodle, en infraestructuras críticas que generan un volumen masivo de datos. Cada interacción del estudiante genera una "huella digital" detallada en los archivos de registro o logs (Soepriyanto, Nugroho, Nahri, Kesuma, & Setiasih, 2025).

A pesar de esta abundancia de información, existe una brecha crítica entre la recolección de datos y su aprovechamiento pedagógico real. El problema se desglosa en tres ejes fundamentales:

- Opacidad del sistema: Los LMS operan frecuentemente como "cajas negras" donde los registros se almacenan sin ser interpretados (Khosravi, y otros, 2022).
- Detección tardía: Los docentes carecen de herramientas para identificar de forma proactiva a los estudiantes en riesgo de fracaso (*Fail*) o abandono (*Withdrawn*), detectando las dificultades cuando ya es demasiado tarde para intervenir.
- Falta de interpretabilidad: Los modelos predictivos avanzados suelen ser complejos y no ofrecen explicaciones claras sobre el "porqué" de una predicción, lo que los hace pedagógicamente estériles para el profesorado.
- Limitación en el modelado temporal: Gran parte de los enfoques actuales analizan el comportamiento del estudiante como una métrica estática acumulada (ej. número total de accesos), ignorando la naturaleza secuencial y evolutiva del aprendizaje. Esta simplificación impide detectar patrones complejos de cambio de comportamiento a lo largo de las semanas, que son indicadores críticos de riesgo.

Se requiere, por tanto, el desarrollo de un sistema inteligente que no solo procese masivamente estos registros para predecir el rendimiento final, sino que lo haga de manera explicable. Es imperativo transformar los datos brutos en "conocimiento accionable" que permita realizar intervenciones personalizadas y fundamentadas en patrones de comportamiento reales (Khosravi, y otros, 2022).

4.2. Solución propuesta.

Al abordar un problema de detección temprana de riesgo, uno de los principales desafíos metodológicos es evitar la "fuga de datos" (data leakage).

Para que el modelo predictivo sea válido y aplicable a la realidad docente, es indispensable garantizar que solo evalúe la información generada desde el inicio del curso (Día 0) hasta el momento exacto en el que se realiza la predicción.

4.2.1. Ventanas de tiempo.

Esta limitación hace necesario introducir el concepto de ventana de tiempo, un enfoque fundamental para estructurar el análisis en este proyecto. Como ya se observó en el Análisis Exploratorio de Datos (EDA), el registro diario de interacciones presenta un nivel de ruido que dificulta la extracción de patrones claros. Sin embargo, al agregar la actividad de los estudiantes de forma semanal, el comportamiento se vuelve mucho más estable y tratable. Esta decisión de diseño concuerda con los hallazgos de Kusumawardani y Alfarozi (2023), quienes demostraron empíricamente que los modelos basados en ventanas semanales tienden a superar a los diarios y resultan computacionalmente más eficientes.

Por consiguiente, de ahora en adelante, la temporalidad del problema se modelará exclusivamente por semanas. En la práctica, estas ventanas de tiempo abarcan desde el inicio del curso hasta la semana W, siendo W la semana exacta en la que deseamos estimar si el estudiante terminará abandonando. Al construir las variables para alimentar al modelo, es vital asegurar que no se filtre ninguna información "privilegiada" del futuro. A modo ilustrativo, si estamos calculando el riesgo de un alumno en la semana 10, resulta evidente que el modelo no puede tener acceso a métricas o calificaciones de exámenes finales que ocurrirán semanas después. Controlar este aspecto de forma estricta es lo que garantiza que nuestra IA funcione como un sistema de alerta temprana.

En cada ventana de tiempo calculamos métricas acumuladas hasta esa semana (clics en los foros, actividades entregadas, notas acumuladas hasta entonces, etc.) y las juntamos con variables que conocemos desde el día cero del

estudiante (rango de edad, índice de riqueza del código postal, interactividad antes del comienzo del curso con la plataforma educativa, etc.)

4.2.2. Reducción de dimensionalidad y Aprendizaje no supervisado.

Una vez obtenida esta información (aproximadamente 58 características), siguiendo una aproximación inspirada en el estudio de El Ghali, Atouf, El Guemmat, Broumi, & Talea (2025), se realiza un agrupamiento (clustering) del estudiantado utilizando exclusivamente la información disponible hasta la semana W, con el objetivo de identificar perfiles de comportamiento diferenciados.

Para optimizar el agrupamiento, se comparan técnicas de reducción de dimensionalidad, concretamente PCA y un autoencoder, con el fin de proyectar la información a un espacio latente de menor dimensión y potencialmente más separable. En el caso del autoencoder se le penalizará no solo por su capacidad de reconstrucción sino también por su capacidad para clusterizar, más visualmente podemos verlo en la siguiente figura:

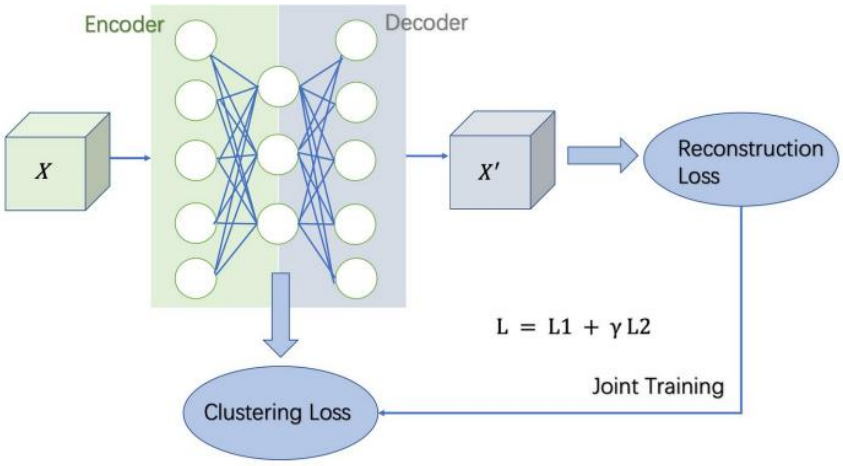

Ilustración 2 Diagrama de Deep Clustering Network (DCN)
https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10047866

En concreto, se entrenará un único autoencoder (común a todas las ventanas) para aprender una representación comprimida de los datos. Posteriormente, se ajustará un modelo de clustering independiente para cada ventana temporal W, de modo que cada uno esté especializado en agrupar a los estudiantes basándose en su comportamiento acumulado desde el inicio del curso hasta la semana W.

El algoritmo de agrupamiento seleccionado para esta fase es el Modelo de Mixtura Gaussiana (GMM, por sus siglas en inglés). La elección se fundamenta en su naturaleza probabilística, la cual se adapta de manera óptima a distribuciones de datos continuas donde las fronteras entre los diferentes perfiles de estudiantes son difusas. A diferencia de los métodos de asignación estricta (hard clustering), GMM no fuerza a un alumno a pertenecer a un único grupo, sino que calcula una distribución de probabilidades. Adicionalmente, desde una perspectiva arquitectónica, alimentar la red neuronal Transformer posterior con un vector continuo de probabilidades (soft labels) resulta mucho más informativo y estable para la optimización del modelo que utilizar un identificador de clase discreto.

4.2.3. Modelado secuencial con Transformer.

El objetivo principal de la red neuronal Transformer es predecir de forma temprana el riesgo de abandono (Withdrawn) o de fracaso académico (Fail). Es importante destacar que la naturaleza del problema predictivo variará en función de la ventana temporal analizada. En etapas tempranas del curso (por ejemplo, a las 5, 10 o 15 semanas), la falta de evaluaciones formales consolidadas limita la capacidad para predecir un suspenso, lo que plantea un escenario de clasificación binaria (aprobar frente a abandonar). A medida que avanza el curso y se recaba mayor evidencia, el problema evoluciona hacia una clasificación multiclase (aprobar, abandonar o suspender). Para gestionar esta dinámica temporal, se diseñará un conjunto de Transformers independientes,

donde cada modelo estará rigurosamente especializado en realizar predicciones utilizando únicamente la información disponible hasta su ventana de tiempo asignada.

Para alimentar estos modelos, se procesarán secuencias temporales que capturen la evolución semanal de la interactividad del estudiante en la plataforma. A esta información secuencial se le integrarán las características derivadas de la fase previa de clustering, concretamente los vectores probabilísticos de pertenencia (soft labels) generados por el GMM. Mediante esta arquitectura híbrida, se busca demostrar empíricamente que el enriquecimiento de los datos secuenciales con representaciones latentes de comportamiento mejora sustancialmente el rendimiento predictivo del Transformer.

4.2.4. IA Explicativa.

Finalmente, para mitigar el problema de la "caja negra" inherente a las arquitecturas de Deep Learning, se abordará la interpretabilidad del sistema integrando técnicas de Inteligencia Artificial Explicable (XAI). El propósito es dotar de transparencia a las predicciones generadas tanto en la fase de clustering como en la clasificación del Transformer, transformando los resultados matemáticos en explicaciones pedagógicas accionables para el docente.

4.2.5. Ecosistema completo

En la siguiente figura se presenta una visión global del ecosistema metodológico desarrollado en este proyecto, mostrando de forma integrada el flujo completo de información desde las fuentes de datos hasta la generación de predicciones y recomendaciones pedagógicas. El diagrama sintetiza las distintas capas del sistema —preprocesamiento e ingeniería de características, aprendizaje no supervisado,

modelado supervisado secuencial y módulos de inteligencia artificial explicable (XAI)— así como las interacciones entre ellas. Esta representación permite comprender cómo cada componente contribuye de manera coordinada al objetivo principal del estudio: la detección temprana del riesgo académico y la generación de explicaciones accionables para el docente.

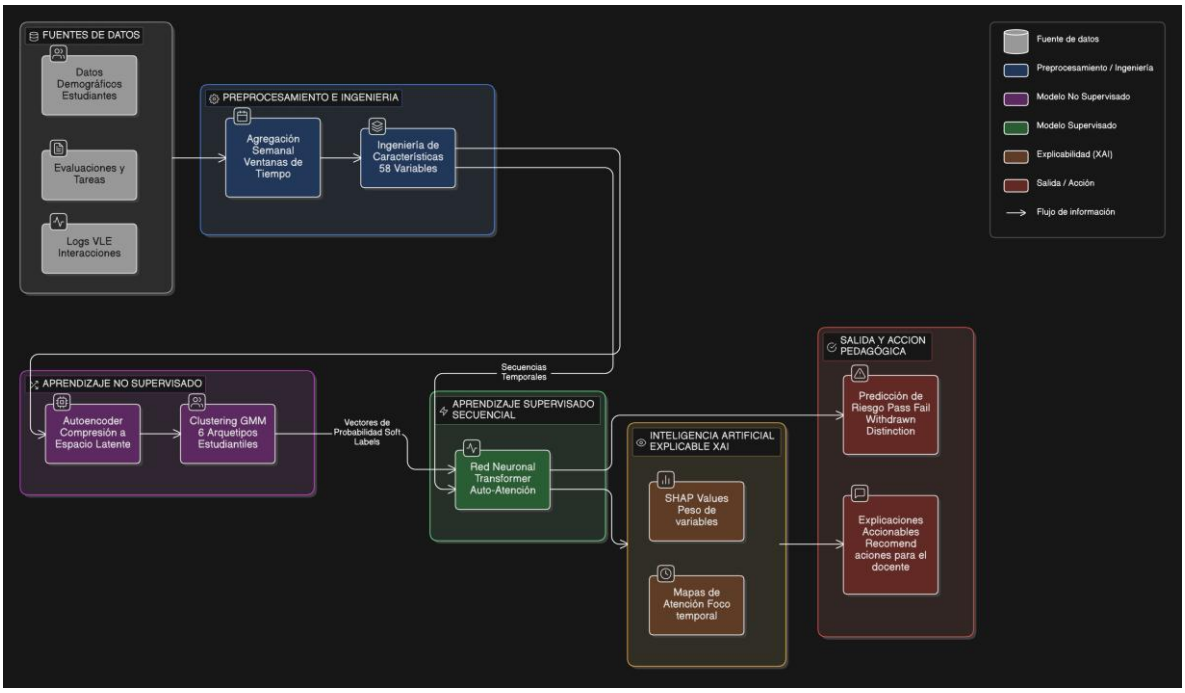

Ilustración 3 Diagrama representativo de la arquitectura del proyecto
Autoría propia - AI Generated

4.3. Metodología.

La implementación de un sistema predictivo basado en *Deep Learning* dentro del contexto educativo requiere un enfoque estructurado que armonice los requisitos técnicos con las necesidades docentes. En este subapartado se detalla el marco metodológico adoptado, el cual permite transitar de manera sistemática desde la fase de comprensión de los datos brutos hasta la generación de acciones pedagógicas concretas y explicables.

Para garantizar la rigurosidad científica y la replicabilidad de la investigación, se describe a continuación el estándar procedimental seguido

(CRISP-DM), la jerarquía de las tareas operativas diseñadas para este estudio junto con las tareas específicas a realizar y la hoja de ruta temporal que guía el desarrollo del proyecto.

4.3.1. Metodología CRISP-DM.

Para transformar los datos brutos de una plataforma educativa en conocimiento útil para un docente, no basta con aplicar algoritmos de forma aislada; es necesario seguir un proceso ordenado y probado. En este trabajo se ha optado por utilizar la metodología CRISP-DM (*Cross-Industry Standard Process for Data Mining*), considerada el "estándar de oro" en los proyectos de ciencia de datos a nivel mundial (Wirth & Hipp, 2000).

A diferencia de un proceso lineal que tiene un principio y un fin rígidos, CRISP-DM es cíclico e iterativo: permite volver atrás y refinar lo aprendido en cada etapa para que el resultado final sea lo más preciso posible. Aunque nació en el ámbito industrial, la flexibilidad de CRISP-DM permite adaptarla perfectamente a las necesidades específicas de este Trabajo de Fin de Máster (TFM). Para este estudio, el ciclo de vida se ha personalizado en seis grandes bloques:

- Entender el problema (Comprensión del Negocio): El primer paso no es programar, sino entender la necesidad pedagógica: ¿Por qué los estudiantes abandonan los cursos? ¿Qué información necesita realmente un profesor para ayudarles a tiempo?
- Explorar los datos (EDA): Analizar la "huella digital" que dejan los alumnos en Moodle para entender qué tipo de información tenemos disponible y qué tan fiable es.
- Preparar la información: Limpiar y organizar esos datos para que las máquinas puedan "leerlos". En este TFM, esto implica convertir los clics

diarios en secuencias que cuentan la historia del estudiante semana a semana. Este enfoque de transformación de datos brutos en nodos de información sigue la tendencia actual de utilizar analíticas internas de los LMS para identificar dificultades de aprendizaje de manera escalable (Soepriyanto, Nugroho, Nahri, Kesuma, & Setiasih, 2025).

- Construir los modelos (Modelado): Aquí es donde introducimos la potencia del Deep Learning. En lugar de usar modelos simples, entrenamos sistemas complejos (*Autoencoders y Transformers*) capaces de aprender patrones de comportamiento que no son obvios a simple vista.
- Verificar y Explicar (Evaluación y XAI): No basta con que el modelo acierte; necesitamos saber por qué lo hace. En esta fase integramos la Inteligencia Artificial Explicable (XAI) para traducir las decisiones matemáticas de la máquina en explicaciones claras para el docente.
- Aplicación de resultados (Despliegue): El paso final consiste en convertir esas predicciones en recomendaciones pedagógicas que puedan ser utilizadas en un entorno real de enseñanza.

Adicionalmente, cabe subrayar que, para este TFM, este bloque no se ha concebido como una serie de pasos manuales, sino como un pipeline de datos automatizado. Esta decisión metodológica garantiza que cualquier transformación (normalización, imputación o ingeniería de variables) se aplique de forma estricta e idéntica a los conjuntos de entrenamiento, validación y test, eliminando el riesgo de "fuga de datos" y asegurando la reproducibilidad total del experimento.

Definida la metodología de trabajo, el siguiente paso consiste en delimitar el alcance del problema técnico y pedagógico. No basta con saber 'cómo' vamos a trabajar (CRISP-DM), sino que debemos precisar 'qué' estamos intentando

resolver y cuáles son los hitos específicos que permitirán alcanzar el éxito del modelo.

4.3.2. Definición de tareas.

Para alcanzar los objetivos propuestos, el proyecto se ha desglosado en una serie de hitos técnicos y operativos. Estas tareas no solo cubren el desarrollo del código, sino también la fundamentación teórica y la transferencia de resultados al ámbito pedagógico:

Fase I: Fundamentación y contextualización

- Comprensión del ecosistema (Business Understanding): Análisis del dominio de la Minería de Datos Educativos (EDM) y la Analítica de Aprendizaje para alinear el modelo con las necesidades docentes.
- Definición de objetivos: Formalización del problema a resolver, estableciendo las métricas de éxito y los límites del sistema predictivo.
- Investigación del estado del arte: Revisión sistemática de literatura científica de los últimos cinco años para identificar arquitecturas de vanguardia y evitar redundancias en la investigación.

Fase II: Gestión y Exploración de Datos

- Adquisición de la fuente de datos: Selección y descarga del conjunto de datos OULAD, por su representatividad en interacciones tipo Moodle.
- Pre-procesamiento y Limpieza: Tratamiento de valores nulos, normalización de variables y filtrado de registros inconsistentes para asegurar la calidad de la entrada.
- Análisis Exploratorio de Datos (EDA): Identificación de patrones visuales, correlaciones y desequilibrios de clase (ej. tasa de abandono frente a suspensos).

Fase III: Ingeniería de Características y Modelado Híbrido.

- Ingeniería de Características I (Transformación Temporal): Conversión de los *logs* de interacción brutos en secuencias temporales estructuradas por semanas de curso.
- Modelado No Supervisado (Clustering): Implementación de algoritmos para descubrir arquetipos de estudiantes de forma automática.
- Ingeniería de Características II (Enriquecimiento): Integración de las etiquetas de los clústeres en el conjunto de entrenamiento para potenciar la capacidad de discriminación del modelo.
- Desarrollo del Modelo Supervisado: Diseño y entrenamiento de una red neuronal basada en la arquitectura Transformer, optimizada para capturar dependencias secuenciales a largo plazo.

Fase IV: Evaluación y Explicabilidad

- Evaluación del desempeño: Cálculo de métricas de rendimiento (Accuracy, F1-Score, Precisión) para validar la robustez de las predicciones.
- Implementación de Inteligencia Artificial Explicable (XAI): Aplicación de técnicas de interpretabilidad para desglosar el peso de cada variable y visualizar los mapas de atención del modelo.
- Acciones Pedagógicas (Transferencia): Traducción de los hallazgos técnicos en recomendaciones concretas para la intervención docente personalizada.

4.3.3. Planificación.

A continuación, se plasman en un diagrama de tipo Gantt la hoja de ruta cronológica y la distribución de esfuerzos diseñada para este trabajo. Este cronograma organiza temporalmente las cuatro fases operativas descritas

anteriormente, estableciendo una secuencia lógica que garantiza el cumplimiento de los objetivos técnicos y pedagógicos dentro del plazo académico establecido.

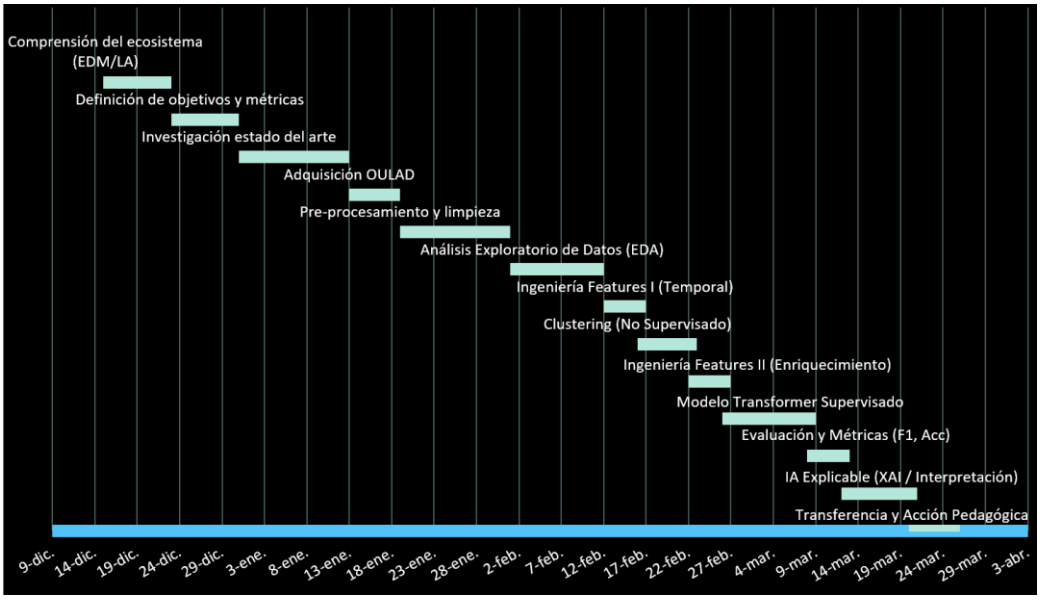

Ilustración 4 Cronograma de tareas definidas. Elaboración propia

5. Desarrollo del proyecto.

La implementación técnica de la arquitectura propuesta se ha llevado a cabo siguiendo los estándares de desarrollo de software y ciencia de datos. Cabe destacar que la totalidad del código fuente, los scripts de preprocesamiento y los cuadernos de experimentación se encuentran alojados en un repositorio público de GitHub (Véase Apéndice I: Repositorio de Código y Reproducibilidad). Esta decisión responde a un doble objetivo metodológico:

- Garantizar la trazabilidad y autoría: El historial de *commits* (registro de cambios) actúa como una prueba temporal veraz del progreso y la originalidad del trabajo realizado, así como la autoría de este.
- Optimizar la memoria académica: Para evitar que este documento se extienda innecesariamente con bloques de código, este apartado se centra en describir la lógica, las decisiones arquitectónicas y los flujos de datos más relevantes.

La estructura de este apartado sigue fielmente las fases del ciclo de vida CRISP-DM, detallando cómo se ha ejecutado cada etapa técnica:

5.1. Comprensión del negocio.

El proceso de comprensión del negocio se inició con una investigación exhaustiva sobre fuentes de datos que cumplieran con requisitos de fiabilidad, volumen y representatividad de un entorno LMS real. Tras evaluar diversas alternativas, se seleccionó el conjunto de datos OULAD (Open University Learning Analytics Dataset). La elección de este dataset se justifica por tres motivos críticos para la calidad de la investigación:

- Veracidad: Contiene datos reales de interacción estudiantil, no sintéticos.

- Anonimización: Cumple con los estándares éticos de privacidad mediante un riguroso proceso de desidentificación.
- Cantidad de datos suficientemente representativa de realidad: El dataset está constituido por 32.593 estudiantes y más de 10 millones de interacciones con el LMS, lo cual, constituye una buena base que representa la realidad.
- Comparabilidad (Benchmarking): Al ser ampliamente utilizado en la literatura científica actual, permite contrastar directamente los resultados obtenidos por nuestra arquitectura frente a otros estudios previos, validando así las mejoras propuestas.

Cabe señalar que la fundamentación teórica del problema, así como el análisis profundo del estado del arte que contextualiza estos datos, se encuentran documentados en detalle en el Capítulo 3 de esta memoria.

Otro aspecto relevante para poder comprender la lógica de negocio del aprendizaje educativo en plataformas es entender los datos de OULAD, observando cómo están estructurados y cómo se relacionan entre sí. El análisis estructural de este conjunto de datos se fundamenta en tres dimensiones que alimentarán el modelo híbrido propuesto:

- Dimensión del estudiante: Consolida la información estática y demográfica, actuando como el perfil base de cada alumno matriculado. Ejemplos: El género del estudiante, su región de residencia, el nivel educativo previo, etc.
- Dimensión de actividades: Contiene los resultados en diferentes actividades realizadas durante el curso por parte del alumno. Ejemplos: La calificación obtenida en una tarea (escala 0-100), el tipo de evaluación (si es corregida por un tutor o automática) y la fecha real en la que el alumno realizó la entrega frente a la fecha límite

- Dimensión de comportamiento: Representa el log de actividad diario en el Sistema de gestión del aprendizaje (LMS) y constituye la huella digital que el estudiante deja al interaccionar con la plataforma. Ejemplos: El número de clics o interacciones diarias con materiales de lectura, la participación en foros de discusión, el acceso a cuestionarios o la descarga de recursos específicos del curso.

El diccionario de datos detallado con la descripción exhaustiva de todas las variables que componen estas tres dimensiones puede consultarse en el Anexo I: Estructura y Diccionario de Datos OULAD. Otros aspectos relevantes de la base de datos son:

- Las fechas normalmente están relativizas con respecto al inicio del curso y se expresan en días previos (negativo) o posteriores (positivos).
- Las interacciones no solo se recogen desde el día cero del curso, sino que antes de iniciar el curso, ya se capturan las interacciones que hacen los estudiantes con la plataforma.

La convergencia entre el análisis de la estructura y relación de los datos con el estudio exhaustivo del marco teórico y el estado del arte constituye el núcleo de la fase de comprensión del negocio. Es importante destacar que, bajo el enfoque iterativo de la metodología CRISP-DM, esta comprensión no es estática; por el contrario, se profundiza y matiza en la comprensión del negocio en cada una de las fases sucesivas del proyecto, desde el preprocesamiento hasta la evaluación de los modelos.

Una vez consolidada esta base conceptual y estratégica, se procede a ejecutar el análisis exploratorio de datos (EDA). El propósito de esta etapa es validar empíricamente la estructura y las interacciones de los datos, permitiendo identificar patrones, sesgos o anomalías que fundamentarán las decisiones técnicas de las fases posteriores

5.2. Análisis exploratorio de datos (EDA).

A nivel técnico, la exploración de los datos (EDA) se materializó mediante scripts de análisis en un jupyter notebook, concretamente en el archivo I_data_exploration.ipynb. Este cuaderno, cuya ubicación exacta dentro de la estructura del proyecto se detalla en el Apéndice I: Repositorio de Código y Reproducibilidad, permite examinar las distribuciones de las variables y la correlación entre las interacciones y el rendimiento final. Del análisis de datos exploratorio (EDA) realizado en dicho cuaderno, se extrajeron las siguientes conclusiones más relevantes:

- Descarte por ética y directrices del EU AI act (Parlamento Europeo y Consejo de la Unión Europea, 2024): Para hacer un sistema ético, equitativo y que cumpla la legislación vigente en la unión europea, datos sensibles como género y discapacidad se descartan del estudio.
- Valores nulos: En el conjunto de datos OULAD, los valores nulos poseen un alto significado estructural y rara vez responden a deficiencias de calidad. Los casos en los que se detectan nulos son:
  - Dimensión de Estudiantes: Se detectaron nulos masivos en la variable date_unregistration (22,521 registros), lo cual es un indicador positivo de persistencia, ya que representa a los alumnos que no han abandonado el curso. Asimismo, variables como imd_band (1,111 nulos) y date_registration (45 nulos) requieren un tratamiento diferente para evitar sesgos de selección al eliminar registros.
  - Dimensión de Evaluaciones: Se identificaron 173 registros sin calificación (score) y 2,865 sin fecha de entrega definida. Estas inconsistencias en los hitos evaluativos marcan la necesidad de un

filtrado estricto en la fase de preprocesamiento para garantizar la veracidad de las etiquetas de rendimiento.

- Dimensión de Interacciones: Los campos temporales en los logs muestran nulos que deben ser normalizados para no interrumpir el cálculo de métricas secuenciales como el momentum o el engagement pesado.
- Balanceo de clases y representatividad: El análisis de la Ilustración 3 revela una distribución asimétrica en los resultados académicos. Si bien existe una base robusta de datos para las categorías de 'Aprobado' y 'Abandono', se observa un desbalance significativo en el grupo de 'Sobresalientes' y una menor representación de 'Suspensos'. Esto nos hace ver que hay que tener especial cuidado con el riesgo de sesgo hacia las clases mayoritarias, lo que justifica técnicamente la elección de métricas de evaluación más precisas que la exactitud simple (accuracy), como el F1-Score, y el uso de arquitecturas de Deep Learning (Autoencoders y Transformers) capaces de aprender representaciones latentes más ricas para identificar patrones sutiles en los grupos con menor representación.

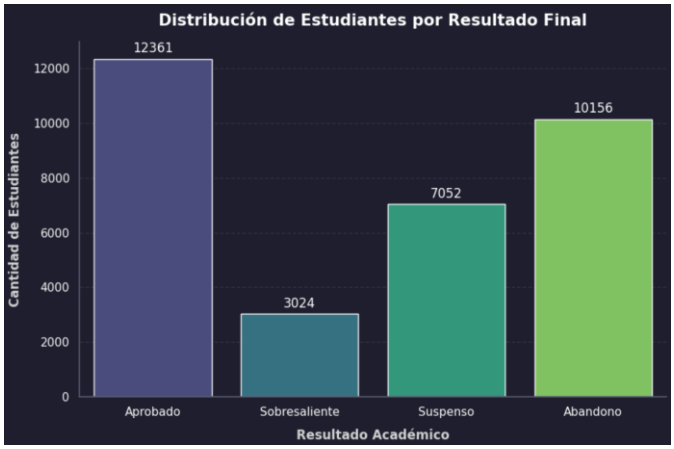

Ilustración 5 Distribución de estudiantes por resultado académico.
Autoría propia - Apéndice 1 - Notebook I

- Ruido diario de interacciones diarias frente a ruido semanal: Al contrastar el promedio de interacciones, se observó que el análisis diario presenta un ruido de alta frecuencia difícil de modelar. Por el contrario,

la agregación semanal ofrece una señal limpia con patrones identificables. Esta observación justifica la decisión técnica de transformar los registros brutos en secuencias semanales para mejorar la capacidad predictiva del modelo.

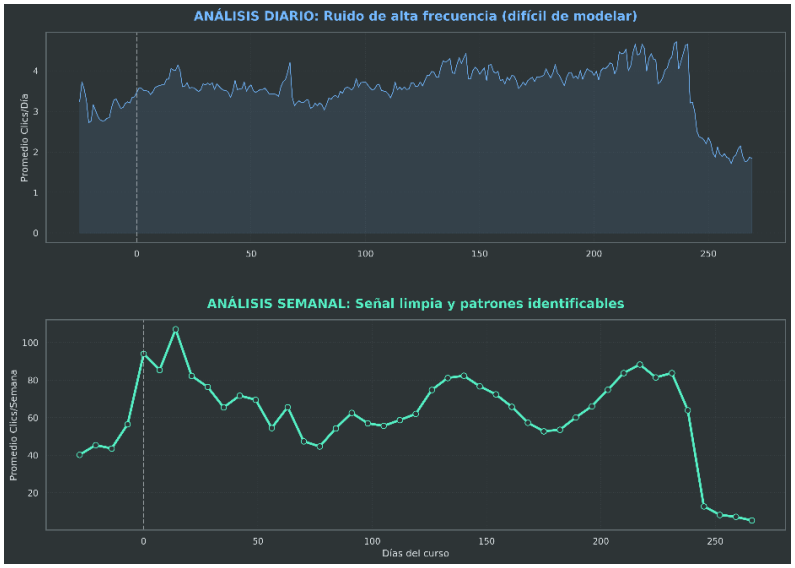

Ilustración 6 Análisis comparativo de de interacciones: Ruido diario frente a señal semanal agregada. Autoría propia - Apendice I - Notebook I

- Nivel de riqueza en el código postal del estudiante: Se observa una relación directa entre el nivel de riqueza medio del código postal del estudiante y los resultados. Vemos como la tasa de abandono y suspensos es más alta en barrios pobres, frente a una mayor tasa de aprobados y sobresalientes en barrios más adinerados.

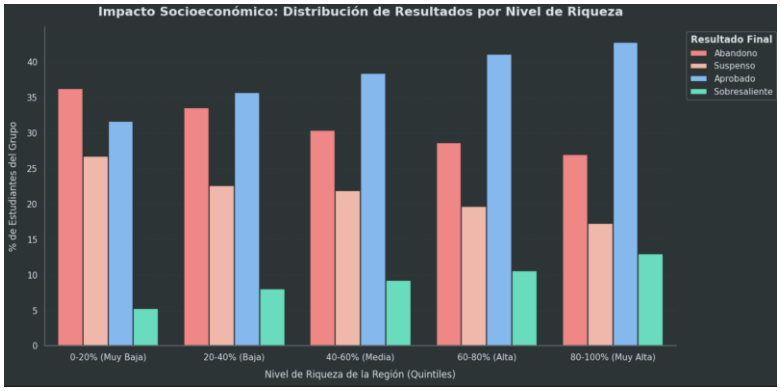

Ilustración 7 Contraste entre riqueza y resultados académicos - Autoría Profia - Apéndice I - Notebook I

- La influencia del tipo de curso en la cantidad de iteraciones: Como muestra el análisis diferencial (ver imagen), la intensidad de interacción varía drásticamente según la materia; por ejemplo, el curso FFF triplica la actividad del BBB. Sin un ajuste previo, el algoritmo de clustering sufriría un sesgo de volumen, agrupando a los estudiantes por la asignatura matriculada en lugar de por su comportamiento real, lo que invalidaría la identificación de patrones de compromiso genuinos.

Para evitar esto, en lugar de contar clics totales, se mide qué tanto destaca un alumno respecto al promedio de su propia clase. Así, podemos comparar con justicia a estudiantes de distintos cursos, asegurando que el análisis se centre en el ritmo y la constancia individual, sin importar si la materia exige muchas o pocas interacciones.

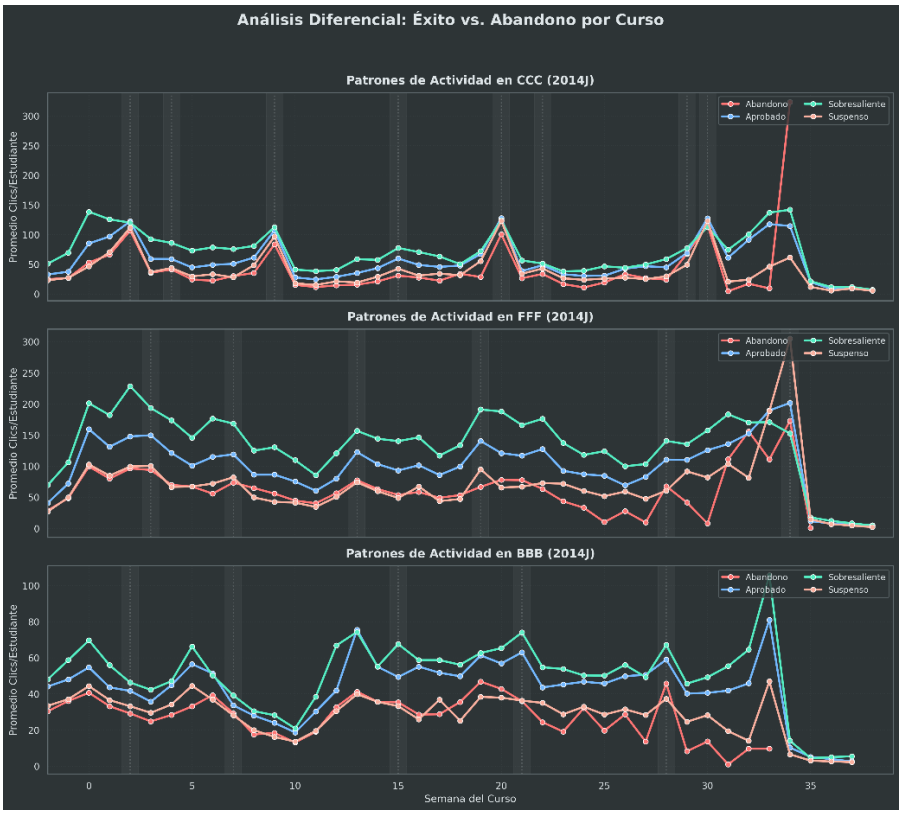

Ilustración 8 Comparación de interacciones por curso. Autoría propia - Apéndice I - Notebook I

- Si nos fijamos en la siguiente ilustración podremos apreciar que no hay relación entre el día que se matricularon los alumnos y si se abandona o si se aprueba, por el contrario, el número de intentos previos y el número de interacciones con la plataforma educativa antes del comienzo del curso sí que están directamente relacionada con la tasa de abandono, el porcentaje de aprobados y el número de sobresalientes, por lo tanto, se las proporcionaremos a nuestro modelo:

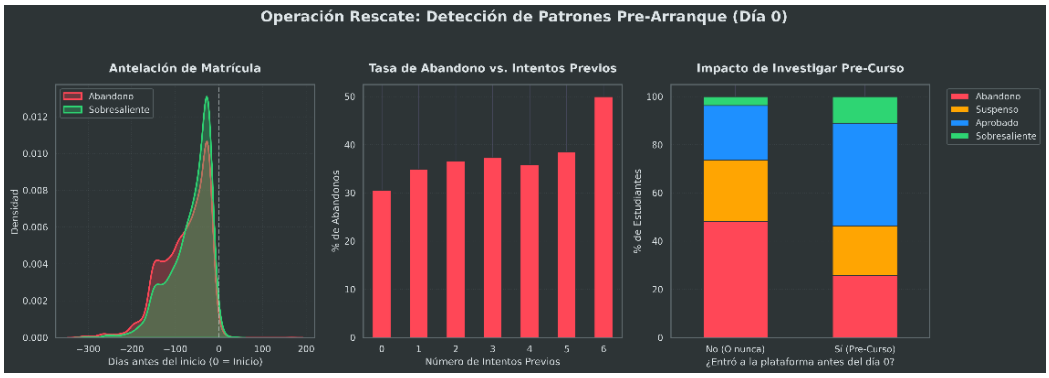

Ilustración 9 Relación entre diferentes métricas y resultados académicos. Apéndice 1 - Notebook I

- Se puede ver también que hay una relación entre el nivel educativo previo y los resultados obtenidos. Cuanto mayor es el nivel educativo previo más probabilidades hay de que el estudiante tenga éxito.

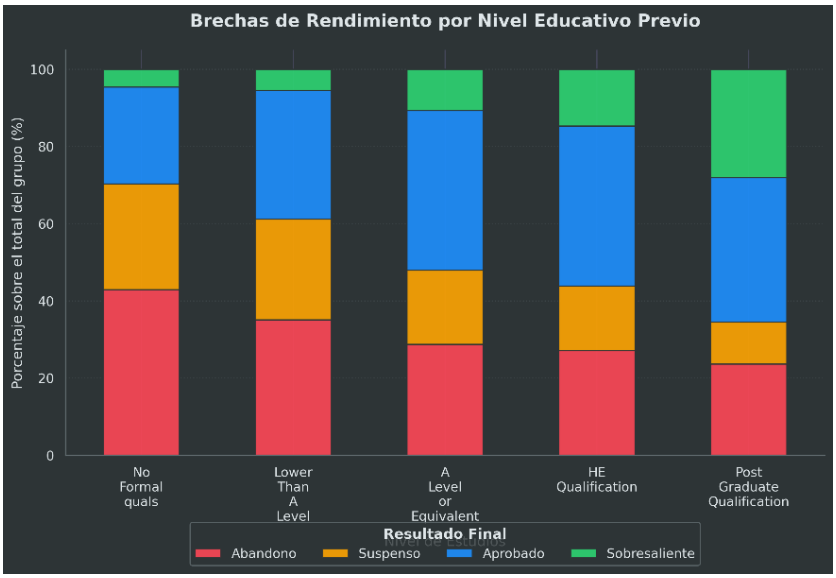

Ilustración 10 Relación entre nivel educativo y resultados académicos
Apéndice 1 – Notebook I

Es fundamental señalar que esta fase de análisis exploratorio (EDA) no constituye un bloque estático o aislado dentro del flujo de trabajo. Por el contrario, el EDA actúa como un proceso transversal y evolutivo que acompaña al ciclo de vida del proyecto. A medida que se itera en las fases de pre-procesamiento, entrenamiento de modelos y evaluación, los hallazgos obtenidos permiten volver al análisis de los datos para pulir métricas y perfeccionar la ingeniería de características.

5.3. Preprocesamiento de datos.

Una vez comprendida la naturaleza y distribución de los datos, se procedió a su preprocesamiento. En primera instancia, se llevó a cabo un tratamiento básico de la información para facilitar el Análisis Exploratorio de Datos (EDA). Posteriormente, el flujo de trabajo se bifurcó en dos enfoques metodológicos: el primero consistió en el preprocesamiento y la ingeniería de características orientados a un modelo Autoencoder, con el objetivo de aplicar técnicas de clustering sobre su espacio latente; el segundo abordó un proceso

análogo, pero adaptado a la red neuronal Transformer. Por consiguiente, la presente subsección se dividirá en el preprocesamiento destinado al EDA, al Autoencoder y, finalmente, a la arquitectura Transformer."

5.3.1. Preprocesamiento para EDA.

El procesado de los datos para la exploración de dato analítica se llevó a cabo a partir de los datos obtenidos de la base de datos oficial de OULAD, el tratamiento de los datos se llevó a cabo en el script dataset.py dentro de la carpeta "educational_ai_analytics/0_data" (Véase apéndice I) y las salidas tras procesar los datos se genera en "data/1_interim" y en "data/2_processed". Las acciones específicas que se llevan a cabo en este script son:

- Consolidación de los múltiples archivos tabulares del sistema (estudiantes, cursos, registros, evaluaciones e interacciones VLE) en tres grandes conjuntos de datos unificados. Durante este proceso, se estandarizaron los valores faltantes o desconocidos (originalmente representados con caracteres especiales como '?').

- Partición y protección contra fuga de datos (Data Leakage): División estratificada de la población estudiantil en conjuntos de Entrenamiento (70%), Validación (15%) y Prueba (15%).

- Durante este preprocesamiento, se prestó especial atención a la imputación de valores nulos para no distorsionar la muestra. Concretamente, las ausencias en el índice de privación múltiple (imd_band) se recategorizaron bajo la etiqueta 'Unknown' para preservar los registros sin introducir sesgos poblacionales. Por su parte, los valores faltantes en la fecha de matriculación (date_registration) se imputaron estadísticamente utilizando la mediana temporal del curso correspondiente

(code_presentation), calculada estrictamente sobre el conjunto de entrenamiento para evitar fuga de datos (data leakage).

5.3.2. Preprocesamiento para Autoencoder y clustering.

La preparación de los datos para el Autoencoder se llevó a cabo en dos scripts de Python, específicamente "ae_uptow_features.py" y "day0_static_features" dentro de la carpeta "educational_ai_analytics /1_proccesed" y salida de estos scripts se generó en la carpeta "data/2_features" (Véase Apéndice I). Aunque a nivel práctico se hayan fusionado el preprocesado e ingeniería de características en un mismo script. En este epígrafe nos limitaremos a la parte de preprocesamiento.

- Para evitar la redundancia y garantizar la trazabilidad longitudinal del alumno, se construyó un identificador único compuesto por el código del estudiante, el módulo matriculado y el semestre (id_student_code_module_code_presentation). Este identificador actuó como índice principal para todas las tablas.

- Normalización condicional por curso: Para evitar que las variaciones inherentes a cada asignatura distorsionaran el análisis del comportamiento, se aplicó una normalización estadística (Z-score) sobre las variables continuas. Esta normalización no se calculó de forma global, sino de manera agrupada y específica para cada cohorte (curso y convocatoria), asegurando que el comportamiento de un estudiante se evalúe exclusivamente en relación con el promedio de sus compañeros de clase.

- Gestión de variables categóricas y valores atípicos (Demografía): Variables ordinales como el índice de privación múltiple (imd_band), el rango de edad (age_band) y el nivel educativo previo

(highest_education) se mapearon a escalas numéricas continuas o discretas, facilitando su interpretación por parte de las redes neuronales.

Adicionalmente, para la imputación de valores faltantes en estas variables categóricas, se aplicó una estrategia basada en la moda estadística del conjunto de entrenamiento.

- Tratamiento robusto de interacciones (Clipping): Dada la alta varianza en la frecuencia de interacción diaria de los estudiantes con la plataforma (clics), se implementó una técnica de clipping (acotamiento). Se limitaron los valores máximos de actividad diaria utilizando el cuantil superior (0.995) de la distribución, mitigando así el impacto de outliers extremos generados por actividad inusual (por ejemplo, bots de descarga masiva).

5.3.3. Preprocesamiento para red neuronal Transformer.

El preprocesamiento para el modelo Transformer difiere del enfoque tradicional, ya que requiere estructurar la información en secuencias temporales en lugar de vectores planos. Este tratamiento se automatizó para transformar los registros diarios en tensores tridimensionales, ejecutando las siguientes acciones clave:

- Estructuración temporal y de identidad: Se generó un identificador único por estudiante y curso, transformando la granularidad de los registros de días relativos a índices semanales.
- Generación de tensores y máscaras: Los registros se agruparon en un tensor de dimensionalidad (N, T, F) (estudiantes, semanas transcurridas, tipos de actividad). Simultáneamente, se creó una máscara booleana (mask_activity) para indicar explícitamente al mecanismo de atención del

Transformer qué semanas contienen señales de comportamiento real y cuáles son vacíos estructurales por inactividad.

- Normalización contextualizada: Para mitigar la alta varianza en la frecuencia de clics, se aplicó una transformación Log1p seguida de una estandarización (Z-score). Para prevenir la fuga de datos (data leakage), los estadísticos de normalización se calcularon estrictamente sobre el conjunto de entrenamiento, diferenciando por curso y considerando exclusivamente las semanas con actividad.

5.4. Ingeniería de características.

En el caso de la ingeniería de características tenemos dos fases: La primera consistente en la preparación y selección de los datos para el autoencoder, el cual representará la información en un espacio latente de menor dimensión que luego será clusterizado. La segunda es dependiente de la resolución de la primera fase, dado que los resultados de las agrupaciones de estudiantes serán inyectados a la entrada de la red neuronal transformer.

Las exploraciones

5.4.1. Fase I: Características para Autoencoder.

El conjunto de características que alimenta el entrenamiento del modelo se construye a partir de los scripts ae_uptow_features.py y day0_static_features.py, alojados en la carpeta educational_analytics/1_features/ (véase el Apéndice I). Una vez procesados los datos provenientes del directorio data/2_processed, las variables resultantes para el autoencoder se exportan a data/3_features. La arquitectura de estos datos se fundamenta en dos bloques principales:

- Información que se tiene del estudiante desde el comienzo del curso, es decir desde el día cero:
  - Datos sociodemográficos: Con el fin de cumplir con aspectos éticos y legales, se descartó incluir información como discapacidad o género. Sin embargo, dado el poder discriminativo que se encontró en EDA de variables como Región, índice de riqueza del código postal, nivel educativo previo, número de veces que intento previamente el curso, etc. Se opto por incluirlas
  - Características de Arranque (Pre-inicio del curso): Se modeló el nivel de proactividad antes del comienzo oficial de las clases (Día 0). Para ello, se calcularon métricas como la intensidad previa (prestart_intensity), que mide la relación entre los clics totales y los días activos, y la anticipación (prestart_anticipation), que cuantifica cuántos días antes del inicio el alumno comenzó a interactuar con la plataforma.
- Información acumulada en diferentes ventanas de tiempo: El objetivo es que para diferentes ventanas de tiempo (desde la semana cero hasta la semana X) se calcules los valores acumulados de clics en los diferentes tipos de actividad, junto con calificación obtenida hasta la fecha, etc.
  - Métricas de Compromiso Ponderado (Weighted Engagement): Se descartó el uso del conteo simple de clics totales, ya que introduce un sesgo de volumen que no refleja necesariamente el aprendizaje real. En su lugar, se creó una métrica de compromiso ponderado (total_weighted_engagement) donde se asignó un mayor peso (1.3) a actividades que requieren esfuerzo cognitivo activo (como cuestionarios o herramientas

colaborativas) y un peso menor (0.8) a la lectura pasiva de recursos.

- Dinámicas Temporales y Momentum: Para capturar el ritmo y la constancia del estudiante, se derivaron variables secuenciales críticas:
  - Tendencia de esfuerzo (effort_slope): Indica si la actividad del alumno va en aumento o en declive.
  - Momentum a corto plazo (momentum): Cuantifica la aceleración o desaceleración del compromiso.
  - Rachas y latencia: Se extrajeron métricas como la racha final de semanas consecutivas activas (streak_final) y la cantidad de semanas transcurridas desde la última actividad (weeks_since_last_activity), fundamentales para detectar riesgo inminente de abandono.
  - Diversidad y Dispersión del Aprendizaje: Se diseñaron indicadores para perfilar el estilo de navegación del alumno. Entre ellos destaca la entropía temporal (temporal_entropy_uptoW), que diferencia a los estudiantes constantes de aquellos que concentran su actividad en picos esporádicos. Asimismo, métricas como el top1_share (proporción de clics en la actividad principal) y el índice de curiosidad (curiosity_index) permitieron medir si el estudiante explora transversalmente los recursos o se limita a un único tipo de material.
  - Evolución del Rendimiento (Assessments): Se generaron características de evaluación que incluyen la ratio de aprobados (pass_ratio), la tendencia en las calificaciones (score_slope), la proporción de entregas tardías (late_ratio) y el tiempo

transcurrido desde la última entrega (weeks_since_last_submission). Adicionalmente, se formuló un índice compuesto (api_index) que pondera la nota media lograda por el logaritmo del volumen de entregas realizadas.

La concatenación de todas estas 58 características conformó el vector de alta dimensionalidad que alimentó la arquitectura del Autoencoder. Esto permitió a la red comprimir la información en un espacio latente, extrayendo patrones de comportamiento no lineales, limpios de ruido y altamente accionables desde la perspectiva pedagógica. Para una revisión exhaustiva del conjunto de datos resultante, el Apéndice II detalla las 58 características definitivas con las que se entrenó el modelo.

5.4.2. Fase II: Características para Transformers.

La segunda fase de la ingeniería de características tiene como propósito construir el conjunto de datos que alimentará el modelo secuencial. A diferencia del Autoencoder, que evalúa vectores estáticos de datos agregados, la red neuronal Transformer requiere una representación dinámica enriquecida con el contexto global del estudiante.

Esta fase de implemento en el script transformer_GLU_classifier.py dentro de la carpeta educational_ai_analytics/modeling/transformes, la exploración previa para modelar este script se encuentra en el notebook 5 (Apéndice I). La arquitectura de las características generadas en este script se estructura en los siguientes tres bloques fundamentales:

- Matriz Secuencial de Interacciones (Comportamiento Temporal): Constituye el núcleo dinámico del modelo. En lugar de métricas acumuladas, la información se estructura en un tensor tridimensional de dimensionalidad (N x T x F) (estudiantes, semanas transcurridas, tipos de

actividad). Esto permite a la red evaluar la evolución pura de las interacciones semanales del alumno con cada recurso del LMS (foros, cuestionarios, lecturas) de forma cronológica.

- Representaciones Latentes y Arquetipos (Soft Labels): En este punto se materializa la arquitectura híbrida del proyecto. A la secuencia temporal se le inyectan las probabilidades continuas de pertenencia de cada estudiante a cada uno de los clústeres generados previamente por el Modelo de Mixtura de Gaussianas (GMM). Alimentar al Transformer con esta distribución de probabilidades (soft labels), en lugar de una clase discreta, proporciona al modelo un contexto rico sobre el arquetipo de aprendizaje del estudiante, estabilizando su optimización. También se incorporan métricas complejas derivadas en la Fase I, como la entropía temporal.
- Características Estáticas y Perfil de Arranque (Día 0): Para contextualizar el punto de partida del estudiante, se concatena al tensor la información disponible antes de iniciar el curso. Esto incluye las variables sociodemográficas de alto poder predictivo (como el índice de riqueza del código postal, el nivel educativo previo y los intentos anteriores) junto con las métricas de proactividad previas al arranque.
- Variables acumulativas en la ventana de tiempo: Como resultado de la fase de experimentación, se optó por incorporar también las características históricas acumuladas por el estudiante hasta la semana objeto de predicción (W). La inyección de estas métricas agregadas —calculadas inicialmente para el Autoencoder— complementa el análisis secuencial del mecanismo de atención, lo que se tradujo empíricamente en una mejora sustancial de las métricas de rendimiento del modelo final.

La integración conjunta de estos tres bloques permite al Transformer no solo procesar "qué" hace el estudiante semana a semana mediante su mecanismo de

autoatención, sino contextualizarlo bajo "quién" es y a qué perfil de comportamiento general pertenece, maximizando así la precisión en la detección temprana del riesgo.

5.5. Modelado

La fase de modelado constituye el núcleo analítico y predictivo de esta investigación. Para abordar la naturaleza multidimensional del comportamiento estudiantil, se ha diseñado una arquitectura híbrida secuencial. Esta aproximación se divide en dos grandes etapas operativas: una primera fase de aprendizaje no supervisado (mediante Autoencoders y clustering) orientada a la reducción de dimensionalidad y el descubrimiento de arquetipos latentes; y una segunda fase de aprendizaje supervisado (basada en redes neuronales Transformer) especializada en modelar la evolución temporal del alumno para predecir su riesgo académico.

A continuación, se detallan los fundamentos técnicos, la experimentación y las decisiones arquitectónicas que sustentan cada uno de estos modelos.

5.5.1. Autoencoder.

El modelado del Autoencoder(AE) se materializa en los scripts autoencoder.py donde se construye la arquitectura del AE con Keras y train_autoencoder.py donde se especifica la manera en la que se entrena, todos estos ficheros se encuentran en la carpeta “educational_ai_analytics/2_modeling/ae” (véase apéndice I).

La estrategia principal consiste en entrenar un único autoencoder global que procesa simultáneamente todas las ventanas semanales (5, 10, 15, 20 y 25). Esta decisión se fundamenta en tres motivos técnicos: primero, garantiza la coherencia del espacio latente, creando un marco de referencia universal que

permite rastrear la evolución del estudiante sin las incompatibilidades de usar modelos independientes. Segundo, actúa como una técnica implícita de aumento de datos (*data augmentation*), multiplicando las muestras disponibles y reduciendo el sobreajuste (*overfitting*). Finalmente, fomenta la extracción de arquetipos universales, obligando a la red a identificar patrones de interacción como el *momentum* independientemente de la semana en la que ocurran, delegando el análisis estrictamente secuencial al modelo Transformer.

Adicionalmente, la arquitectura implementada se enmarca en la categoría de redes Deep Clustering Networks (DCN), inspirada en los principios del Deep Embedded Clustering (DEC). A nivel de optimización, esto significa que el modelo no se limita únicamente a la reducción de dimensionalidad, sino que es guiado por una función de pérdida dual durante su fase de entrenamiento.

Por un lado, la red minimiza una pérdida de reconstrucción (empleando la métrica Huber Loss) para garantizar que el espacio latente retenga fielmente la información estructural y las características originales del estudiante. Por otro lado, incorpora simultáneamente una pérdida de agrupamiento basada en la divergencia de Kullback-Leibler (KL). Mediante esta optimización conjunta, el autoencoder genera embeddings que no solo están limpios de ruido, sino que están matemáticamente forzados a ser altamente discriminativos para la segmentación de perfiles de aprendizaje.

Para llevar a cabo esta optimización dual de manera estable, el proceso de entrenamiento del modelo (implementado en el script train_autoencoder.py) no se ejecuta en un solo paso, sino que se estructura cuidadosamente en tres fases secuenciales:

- Fase 1: Pre-entrenamiento (Reconstrucción): En esta etapa inicial, la red se entrena de manera estándar como un autoencoder tradicional,

desactivando temporalmente el objetivo de clustering. El objetivo exclusivo es minimizar el error de reconstrucción mediante la métrica Huber Loss. Esto garantiza que el modelo aprenda primero a proyectar los datos en un espacio latente que retenga fielmente toda la información estructural (estática y dinámica) de los estudiantes, construyendo una base sólida y libre de ruido.

- Fase 2: Inicialización de Centroides (K-Means): Una vez obtenido un espacio latente pre-entrenado y estable, se extraen los embeddings de los estudiantes y se aplica sobre ellos el algoritmo K-Means clásico. Los centroides encontrados (k=X, según lo determinado en la experimentación) no se usan como resultado final, sino que se inyectan en la red neuronal para inicializar los pesos de la capa específica de clustering. Este paso es un requisito clave del enfoque DEC, ya que proporciona a la red un punto de partida óptimo antes de aplicar la pérdida probabilística.
- Fase 3: Optimización Conjunta (Entrenamiento DCN): Es la fase central y más compleja del modelado. Aquí, la red se afina (fine-tuning) minimizando simultáneamente la pérdida de reconstrucción y la divergencia de Kullback-Leibler (KL). Durante esta etapa, el modelo calcula una distribución objetivo (P) matemática que fuerza a las predicciones probabilísticas a ser más "nítidas" (acercando los estudiantes a su centroide más próximo y alejándolos del resto). Esta distribución se recalcula iterativamente a lo largo de las épocas, guiando al espacio latente para que aumente la cohesión intra-clúster sin perder la capacidad de reconstrucción original.

Adicionalmente, a nivel de ingeniería de Machine Learning, el bucle de entrenamiento incorpora técnicas avanzadas para garantizar la robustez del modelo, tales como Early Stopping basado en la pérdida conjunta para evitar el

sobreajuste (overfitting), y el uso de precisión mixta (mixed_float16) para optimizar drásticamente los tiempos de cómputo en la GPU.

5.5.2. Embeddings

Los embeddings —entendidos como representaciones comprimidas de los vectores de datos de los estudiantes en un espacio latente de menor dimensionalidad— fueron creados empleando el autoencoder descrito en el epígrafe anterior. De forma complementaria, y con el objetivo de disponer de una línea base (baseline) interpretable, se entrenó también un modelo PCA, ajustando su número de componentes al mismo tamaño dimensional del espacio latente del autoencoder. Esto nos permite reducir la dimensionalidad de los datos siguiendo dos filosofías diferentes (PCA y AE). Esto permite realizar un comparación que evaluará cuál de las dos técnicas resulta más adecuada como etapa previa al clustering.

Para la generación de los embeddings se utilizó el script "generate_embeddings.py" en la carpeta educational_ai_analytics/2_modeling /ae (véase apéndice I), este script toma como entrada los datos en la carpeta "data/3_features" y a partir de ella se generan los embeddings tanto para AE (ae_latent.csv) como para PCA(pca_latent.csv), estos vectores característicos de los estudiantes en el espacio latente son los que se utilizarán en la siguiente fase de Clustering.

En la siguiente ilustración se presenta una representación visual del funcionamiento del autoencoder en tres niveles: (1) el espacio original proyectado mediante PCA, (2) el espacio latente aprendido por el modelo y (3) la reconstrucción nuevamente proyectada en el espacio PCA. Las filas corresponden a distintas ventanas temporales (W05, W10 y W15), lo que permite observar la estabilidad del mapeo aprendido a lo largo del tiempo.

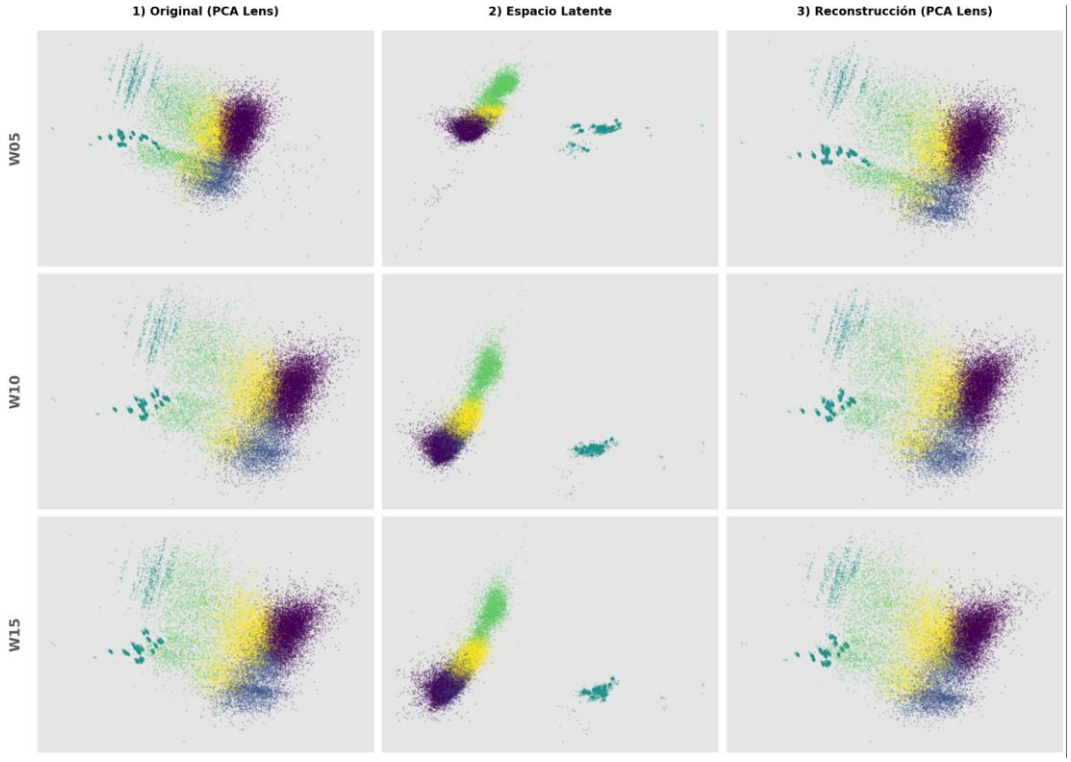

Ilustración 11 Representación de datos de estudiantes en el espacio original, latente y reconstrucción - Autoría Propia - Apéndice I - Reportes -AE

Puede apreciarse cómo, en el espacio latente, las representaciones se compactan y reorganizan en estructuras más definidas y separables, evidenciando el efecto conjunto de la pérdida de reconstrucción y la pérdida de clustering. Al mismo tiempo, la reconstrucción mantiene una alta fidelidad respecto al espacio original, lo que confirma que la compresión no implica pérdida sustancial de información estructural. Este comportamiento valida empíricamente que el autoencoder no solo actúa como un mecanismo de reducción de dimensionalidad, sino como un generador de embeddings estructuralmente coherentes y optimizados para la segmentación, consolidando perfiles de aprendizaje más nítidos y matemáticamente discriminativos.

5.5.3. Clustering

El modelado de clustering requirió una fase experimental iterativa que implicó realizar múltiples ajustes y reentrenamientos a lo largo de las etapas anteriores (procesamiento, EDA, ingeniería de características y modelado). A continuación, se describen dos ejemplos representativos:

- Control de fuga de datos mediante ventanas temporales. La aproximación inicial abarcaba todo el curso en una única ventana, lo que introducía información futura y generaba agrupaciones artificialmente precisas (fuga de datos). Para alinearse con el objetivo predictivo del proyecto —que exige evaluar al estudiante solo con la información disponible hasta un momento dado—, se transicionó hacia un enfoque de ventanas temporales acumulativas. Esto requirió adaptar la ingeniería de características y el autoencoder para generar modelos de clustering específicos por semana K, basados estrictamente en el comportamiento histórico hasta dicha semana.
- Mejora del espacio latente mediante pérdida de clustering. Inicialmente, el autoencoder se entrenaba únicamente minimizando el error de reconstrucción. Aunque esta estrategia lograba compresión, el espacio latente resultante no siempre era fácilmente separable para técnicas de agrupamiento. Al incorporar una pérdida adicional basada en la divergencia de Kullback–Leibler (KL) —siguiendo el enfoque DEC/DCN— el modelo empezó a producir embeddings más clusterizables, aumentando la cohesión intra-clúster y facilitando una segmentación más clara en la fase de clustering.

En conjunto, estos ejemplos reflejan que el desarrollo no siguió un flujo estrictamente lineal, sino un proceso iterativo, en el que cada decisión en una fase impactaba directamente en las siguientes, requiriendo ajustes sucesivos

hasta converger en una solución metodológicamente consistente con el objetivo de alerta temprana.

El detalle completo de los experimentos para tomar decisiones de clustering puede consultarse en el Notebook 4 dentro de la carpeta Notebooks del repositorio (véase Apéndice I). En dicho notebook se exploran dos familias de combinaciones: (PCA o AE) + K-Means y (PCA o AE) + Gaussian Mixture Model (GMM). No obstante, para evitar extender en exceso la memoria, en este epígrafe se analiza únicamente la segunda combinación, es decir, PCA o AE con GMM. Esta decisión se alinea con lo expuesto y justificado en el epígrafe anterior: por un lado, la naturaleza del problema no sugiere fronteras "duras" entre perfiles de estudiantes, y por otro, el uso posterior de modelos basados en Transformers se beneficia de representaciones probabilísticas (p. ej., pertenencias blandas o distribuciones), lo que hace al GMM más coherente con el enfoque global.

En la figura siguiente se muestra la evolución de los criterios de información BIC/AIC para distintos valores de k (número de componentes) al aplicar GMM sobre embeddings generados mediante PCA y mediante autoencoder. A partir de esta comparativa se puede observar el comportamiento relativo de ambas técnicas en términos de ajuste y parsimonia del modelo.

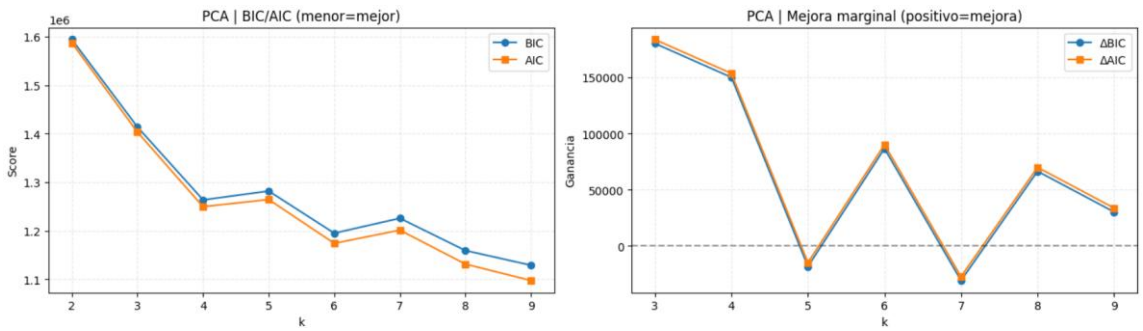

Ilustración 12 Comparación BIC/AIC en PCA
Autoría propia - Apéndice I - Notebook 4

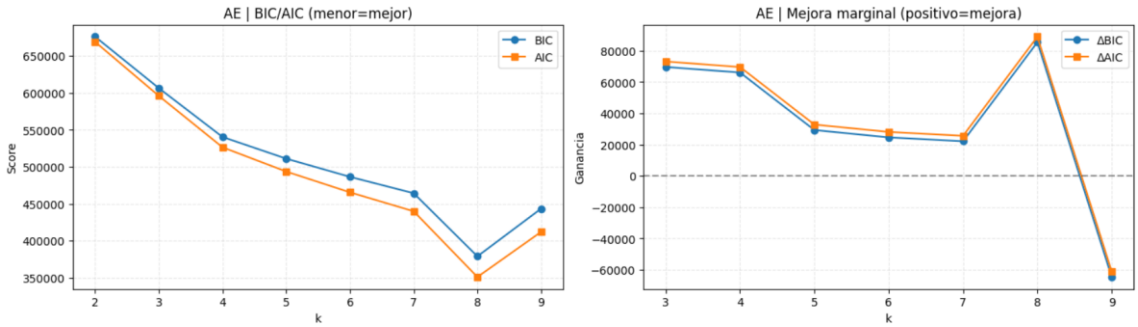

Ilustración 13 Evolución BIC/AIC con AE - Autoría propia - Apéndice I - Notebook 4

Para evaluar la calidad de estas agrupaciones, se emplean los Criterios de Información Bayesiano (BIC) y de Akaike (AIC). Ambas métricas buscan encontrar el modelo más eficiente: miden qué tan bien se ajustan las agrupaciones a los datos reales, pero aplican una penalización matemática si el modelo se vuelve demasiado complejo (en este caso, si se añaden demasiados clústeres innecesarios). Por consiguiente, en ambas métricas, un valor más bajo indica siempre un modelo mejor (mayor equilibrio entre precisión y simplicidad).

Es importante destacar que, para garantizar una comparativa rigurosa y equitativa, los embeddings generados tanto por PCA como por el Autoencoder (AE) fueron previamente estandarizados. Esta transformación permite que ambos espacios latentes compartan la misma escala y, por tanto, que la magnitud de los criterios de información sea directamente comparable entre los dos enfoques.

Al observar las gráficas, la primera diferencia notable y concluyente radica en la escala absoluta de los resultados. En la Ilustración 12 (PCA), los valores de BIC y AIC se sitúan en el orden de los millones (por encima de 1.1×10^6). En claro contraste, la Ilustración 13 (AE) muestra valores significativamente inferiores, todos por debajo del millón (oscilando entre 350.000 y 650.000). Esta drástica diferencia de magnitud evidencia, de entrada, que las representaciones extraídas por el Autoencoder logran un ajuste global muy superior al de PCA.

Además de la superioridad en la escala absoluta, la evolución de los criterios a medida que aumenta k revela comportamientos distintos. En el caso de PCA, la gráfica de mejora marginal muestra fuertes oscilaciones e inestabilidad, llegando a registrar empeoramientos (valores negativos) en k=5 y k=7. Esta volatilidad sugiere que la reducción de dimensionalidad lineal no logra desenredar adecuadamente la complejidad de los datos, dificultando que el algoritmo GMM encuentre agrupaciones naturales y estables.

Por el contrario, la curva de los valores para el Autoencoder desciende de manera mucho más suave, progresiva y consistente. Al examinar su gráfica de mejora marginal, se constata que las ganancias se mantienen estables y positivas a medida que se añaden componentes, alcanzando su pico máximo de mejora en k=8. Sin embargo, al probar con k=9, se produce una caída drástica y pronunciada hacia valores negativos. Esto señala un claro punto de inflexión: añadir un noveno clúster penaliza fuertemente al modelo por exceso de complejidad, sin aportar una mejora en el agrupamiento que lo justifique (riesgo de sobreajuste).

A la luz de estos resultados empíricos, se extraen dos conclusiones fundamentales para la definición de la arquitectura final del sistema:

- Superioridad del Autoencoder para la extracción de características: Las representaciones no lineales generadas por el AE (optimizadas adicionalmente con la pérdida de clustering) permiten al algoritmo GMM encontrar una estructura de agrupamiento mucho más definida, robusta y con un ajuste matemáticamente superior al del PCA.
- Selección del número óptimo de clústeres (k): El valor k=X emerge como la configuración óptima para el modelo GMM sobre los embeddings del AE. Representa el "codo" o punto de máximo beneficio en el compromiso entre el ajuste a los datos y la parsimonia del modelo.

Por consiguiente, para la generación de las secuencias de entrada que alimentarán posteriormente al modelo basado en Transformers, se consolida el uso de un modelo de mezclas gaussianas (GMM) de X componentes entrenado sobre el espacio latente del Autoencoder. Esta configuración permite perfilar a los estudiantes en ocho estados probabilísticos distintos cada semana, proporcionando una granularidad rica y matemáticamente fundamentada para la detección temprana de riesgo.

5.5.4. Modelado del Transformers.

Para el modelado de la red neuronal Transformers se uso Keras, el archivo en el que se materializa la implementación del modelo es "transformer_GLU_classifier.py" dentro de la carpeta "ai_educational_analytics/2_modeling/transformer", y su entrenamiento se realiza en el archivo "train_transformer.py", complementariamente hay muchos scripts que sirven de utilidades (finetuning, comparaciones de métricas entre entrenamientos, etc.)

Para la tarea de clasificación final (predecir el éxito, fracaso o abandono del estudiante), se diseñó una red neuronal a medida implementada en Keras. A diferencia de un modelo tradicional que mezcla todos los datos de golpe, esta arquitectura se concibió con un enfoque "multimodal": separa la información en dos grandes ramas paralelas para procesarla de la manera más adecuada antes de tomar una decisión conjunta.

- Procesamiento en paralelo: El modelo recibe la información dividida en dos naturalezas muy distintas, y utiliza una rama especializada para cada una con el objetivo de traducirlas a un espacio latente común:
  - Rama de secuencias temporales: Toma las interacciones del estudiante semana a semana (clics, entregas, tiempo de conexión). Esta

información pasa por una capa de proyección que transforma estas variables brutas en representaciones vectoriales (embeddings) más compactas y ricas en significado.

- Rama de contexto estático: Paralelamente, todas las variables estáticas (datos demográficos, actividad previa al inicio del curso, métricas acumuladas y el perfil probabilístico asignado por el Autoencoder/GMM) se procesan a través de una red neuronal densa clásica. El objetivo de esta rama es destilar el "perfil base" del estudiante y proyectarlo a la misma dimensión que los datos temporales.

- Fusión temprana (Early Fusion): Mediante un mecanismo de compuertas (Gated Fusion), el modelo inyecta el "perfil base" del estudiante en cada una de las semanas de su secuencia temporal. De este modo, la red aprende a interpretar una misma acción (por ejemplo, una bajada de actividad en la semana 3) de forma distinta dependiendo de si el alumno tiene un perfil histórico de alto riesgo o si es un estudiante excelente que simplemente tuvo una semana atípica.

- El motor de atención: La secuencia, ya enriquecida con el contexto estático del alumno, entra en el corazón del modelo: el bloque Transformer. Aquí, los mecanismos de "Atención Multicabezal" (Multi-Head Attention) analizan la secuencia completa de golpe. La red aprende a conectar eventos pasados con comportamientos recientes, entendiendo la evolución del estudiante. Para que este análisis sea preciso, se utilizan máscaras lógicas; estas indican al modelo qué semanas no tuvieron actividad real o cuáles son simplemente relleno (padding), permitiendo que la red ignore el "ruido" y se concentre solo en la información válida.

- Decisión final y fusión tardía (Late Fusion): Tras el análisis secuencial, el Transformer comprime toda la evolución del estudiante en

un único vector de resumen. Para tomar la decisión final con la máxima garantía, el modelo junta tres piezas clave de información:

1. El resumen de la evolución temporal (lo que acaba de calcular el Transformer).
2. El perfil estático procesado.
3. Los datos estáticos crudos originales (un "bypass" directo para que el clasificador no olvide datos cruciales como la demografía exacta).

Finalmente, esta información combinada pasa por un último bloque de red neuronal (el Head de clasificación) que emite la predicción probabilística sobre el estado final del estudiante en el curso.

5.5.5. Estrategia del entrenamiento y configuración de clases.

Una vez diseñada la arquitectura multimodal, el proceso de aprendizaje del modelo (gestionado mediante el script train_transformer.py) se configuró estratégicamente para afrontar uno de los mayores retos de la analítica educativa: el fuerte desbalanceo natural de los alumnos y la necesidad de adaptar el sistema a diferentes definiciones de "riesgo académico".

Para lograr que el modelo sea útil en un entorno real, la estrategia de entrenamiento se fundamenta en los siguientes pilares:

- Flexibilidad en los escenarios de predicción: El sistema no está anclado a un único objetivo. Permite resolver el problema con distintos niveles de granularidad: desde una clasificación binaria directa (estudiantes en riesgo frente a casos de éxito, permitiendo aislar abandonos o suspensos según convenga), pasando por una visión trinaria (fracaso, abandono y éxito), hasta una clasificación cuaternaria que

respeta todas las calificaciones originales (incluyendo las matrículas de honor.

- Aprendizaje enfocado en la minoría (Focal Loss): En un curso habitual, la gran mayoría de los alumnos aprueba y solo una minoría abandona. Si el algoritmo se entrenara de forma tradicional, tendería a ignorar a los alumnos en riesgo porque le resultaría más "rentable" predecir siempre que todos van a aprobar. Para neutralizar este sesgo, se emplea una técnica matemática llamada Focal Loss. Este mecanismo penaliza severamente a la red cuando se equivoca en los casos raros (la minoría en riesgo) y le resta importancia a los alumnos que son evidentes, obligando al modelo a concentrarse en los estudiantes más difíciles de detectar.
- Ajuste dinámico de la alerta (Threshold Tuning): En los escenarios de predicción de riesgo, disparar una alerta simplemente porque la probabilidad cruza la barrera del 50% rara vez es la mejor decisión. El sistema incorpora un algoritmo que evalúa cientos de puntos de corte de forma automática durante la fase de validación. Su objetivo es encontrar el umbral matemático óptimo que logre el equilibrio perfecto: maximizar la cantidad de alumnos en peligro detectados (exhaustividad), garantizando al mismo tiempo que no se sature a los docentes con falsas alarmas (precisión).
- Estabilidad y prevención de sobreajuste: Para asegurar que el modelo aprenda a deducir patrones reales y no se limite a memorizar el comportamiento de los alumnos de entrenamiento, el sistema monitoriza su propio rendimiento constantemente. Si detecta que el aprendizaje se estanca, reduce de forma autónoma la velocidad a la que actualiza sus conocimientos para realizar ajustes más finos. Si la mejora se detiene por completo, el entrenamiento finaliza prematuramente (Early Stopping) y

rescata la versión exacta de la red que logró el mejor equilibrio predictivo.

Finalmente, para gobernar la enorme complejidad de este ecosistema predictivo y garantizar la robustez de los resultados, el proyecto se dotó de una infraestructura de experimentación automatizada. Conscientes de que el rendimiento de una red Transformer es altamente sensible a su configuración, se desarrolló un script de búsqueda diseñado para evaluar miles de combinaciones de hiperparámetros (tasas de aprendizaje, dimensiones latentes, número de cabezas de atención, ratios de dropout, etc.). Para gestionar la ingente cantidad de modelos generados en este proceso, se implementó paralelamente un sistema de evaluación comparativa automática. Esta herramienta rastrea, archiva y contrasta el rendimiento de cada iteración, permitiendo auditar de un vistazo qué modificaciones se traducen en mejoras reales (aumentos en Recall o precisión) y cuáles provocan un empeoramiento.

Gracias a este marco de trabajo, el ajuste fino del modelo dejó de ser un proceso de ensayo y error manual para convertirse en una toma de decisiones guiada estrictamente por la evidencia empírica.

5.6. Evaluación

A nivel técnico, el proceso de evaluación se ha diseñado de manera transversal a lo largo de los diferentes scripts y cuadernos de experimentación del proyecto. Para garantizar una trazabilidad rigurosa, todos los reportes, gráficas y métricas generadas durante las fases de validación se exportan automáticamente al directorio reports/, mientras que los archivos con los pesos de los modelos finalmente entrenados se preservan en la carpeta models/ (Véase apéndice I).

El presente epígrafe tiene como propósito establecer el marco teórico y justificar la elección de las métricas que rigen esta evaluación, reservando el análisis empírico de las cifras obtenidas para el capítulo de Resultados.

Adicionalmente, la evaluación de un sistema predictivo en el ámbito de la Minería de Datos Educativos no puede limitarse a cuantificar su tasa de acierto global. Dado que el objetivo primordial del proyecto es construir una alerta temprana para permitir la intervención docente, el modelo debe ser evaluado bajo una óptica multidimensional: su robustez estadística frente a datos desbalanceados, el análisis del coste del error y su interpretabilidad pedagógica.

5.6.1. Evaluación de autoencoder.

Dado que el ecosistema propuesto incluye un bloque de aprendizaje no supervisado, su éxito no puede medirse con métricas de clasificación tradicionales. La calidad del Autoencoder y del agrupamiento posterior se evaluará mediante tres enfoques:

- Fidelidad de Reconstrucción: Monitorización de la pérdida de reconstrucción (Huber Loss) durante el entrenamiento, garantizando que la

compresión al espacio latente no destruya la información estructural del estudiante.

- Calidad del Agrupamiento (Criterios de Información): Para evaluar la calidad de los grupos formados, utilizaremos el Criterio de Información Bayesiano (BIC) y el de Akaike (AIC). En términos sencillos, estas métricas buscan un equilibrio entre qué tan bien se ajusta nuestro modelo (el Modelo de Mixtura Gaussiana) a los datos y su complejidad. El objetivo de usar ambas métricas es asegurarnos de encontrar la agrupación más precisa y, al mismo tiempo, lo más simple posible, evitando crear grupos innecesarios.

- Calidad de la Segmentación (Clustering como Downstream Task): La prueba definitiva de la utilidad del Autoencoder no es su capacidad de compresión aislada, sino su impacto en la fase de agrupamiento posterior. Se evaluará empíricamente si el espacio latente generado por el Autoencoder permite al algoritmo GMM encontrar perfiles de estudiantes más definidos y matemáticamente robustos (basándose en los criterios BIC/AIC) en comparación con el uso de reducciones lineales clásicas (PCA).

5.6.2. Evaluación de Clustering.

5.6.3. Evaluación de Transformer.

Si bien los valores de Silhouette obtenidos (~0.32) se sitúan en un rango que la literatura clásica define como estructura débil (Rousseeuw, 1987), autores en el área de Learning Analytics (p.ej. Vellido et al.) sostienen que en datos de comportamiento humano, la superposición de perfiles es inherente. Por tanto, se prioriza la estabilidad temporal observada en las semanas analizada y la

interpretabilidad pedagógica de los 5 grupos sobre la maximización de la métrica de cohesión.

5.7. Infraestructura de Ingeniería: Dockerización y Pipelines Automatizados

Para garantizar que la arquitectura de Deep Learning sea agnóstica al hardware diferentes dispositivos y así garantizar su reproducibilidad se ha diseñado el proyecto bajo una infraestructura modular y reproducible basada en dos pilares:

Contenerización del entorno (Docker): Se ha implementado una arquitectura de contenedores que aísla las dependencias críticas (como las bibliotecas CUDA/cuDNN para aceleración por GPU):

- Dockerfile: Define una imagen base inmutable con la versión exacta de python 3.11. Incluye todas las capas de sistema necesarias para la aceleración por hardware y las bibliotecas de cálculo numérico (TensorFlow, Scikit-learn), garantizando que el modelo se comporte igual en un servidor local que en la nube, o en cualquier hardware.
- Docker-Compose: Actúa como el orquestador del entorno, configurando de forma automática el montaje de volúmenes de datos, la gestión de puertos y la inyección de variables de entorno, permitiendo levantar el "laboratorio experimental" completo con un solo comando.
- Dev Containers: Se ha integrado un estándar de desarrollo que permite vincular el IDE directamente con el contenedor. Esto asegura que el código escrito y el código ejecutado compartan siempre el mismo ecosistema, librerías y versiones de compiladores.

Orquestación del Pipeline (Makefile): A diferencia del enfoque tradicional basado exclusivamente en notebooks experimentales, este proyecto implementa una Pipeline-as-Code profesional. Se ha diseñado un archivo maestro de orquestación (Makefile) que automatiza el flujo de trabajo completo:

- Ingesta de datos (make data): Realiza la limpieza, el limpiado preventivo de directorios y la división del dataset (Split) de forma determinista.
- Entrenamiento y Latencia (make prep_latent): Encadena de forma secuencial el entrenamiento del Autoencoder y la generación de embeddings, asegurando la integridad referencial entre los modelos y sus representaciones vectoriales.
- Reproducibilidad Total: Este sistema permite que cualquier investigador pueda replicar el experimento completo desde cero ejecutando un comando único, eliminando la intervención manual y los posibles sesgos humanos en el preprocesamiento de los datos.

6. Resultados Preliminares

7. Conclusiones y trabajos futuros.

8. Referencias

Al-Tameemi, G., Xue, J., Ali, I. H., & Ajit, S. (2024). A Hybrid Machine Learning Approach for Predicting Student Performance Using Multi-class Educational Datasets. *Procedia Computer Science, 238*, 888-895. doi:10.1016/j.procs.2024.06.108

El Ghali, M., Atouf, I., El Guemmat, K., Broumi, S., & Talea, M. (2025). ENHANCING E-LEARNING THROUGH STRATEGIC STUDENT SEGMENTATION: INSIGHTS FROM THE OULAD DATABASE . *Theoretical and Applied Information Technology, 103*(4), 1290-1299.

Firat, M. (2025). Comparative Analysis of Random Forest vs XGBoost Machine Learning Algorithms for Predicting ODL Student Success.

Gamage, S. H., Ayres, J. R., & Behrend, M. B. (Enero de 2022). A systematic review on trends in using Moodle for teaching and learning. *International Journal of STEM Education, 9*(9), 1-24. doi:https://doi.org/10.1186/s40594-021-00323-x

Gašević, D., Dawson, S., Rogers, T., & Gasevic, D. (2016). Learning analytics should not ignore instructional conditions. *Computers & Education*, 53-66.

Hasan, R., Palaniappan, S., Mahmood, S., Abbas, A., Sarker, K. U., & Sattar, M. U. (2020). Predicting Student Performance in Higher Educational Institutions Using Video Learning Analytics and Data Mining Techniques. *Applied Sciences, 10*(11), 3894. doi:10.3390/app10113894

Instituto Andaluz Interuniversitario en Ciencia de Datos e Inteligencia Computacional (DASCI). (s.f.). *DASCI.* Recuperado el 2026, de Minería

de datos educativos y análisis del aprendizaje: https://dasci.es/linea-investigacion/mineria-de-datos-educativos/

Junejo, N. U., Nawaz, M. W., Huang, Q., Dong, X., Wang, C., & Zheng, G. (2024). Accurate Multi-Category Student Performance Forecasting at Early Stages of Online Education Using Neural Networks. *arXiv preprint arXiv:2412.05938*, 1-27.

Khosravi, H., Buckingham Shum, S., Chen, G., Conati, C., Tsai, Y.-S., Kay, J., . . . Gašević, D. (2022). Computers and Education: Artificial Intelligence. *Computers and Education: Artificial Intelligence, 3*. Obtenido de https://doi.org/10.1016/j.caeai.2022.100074

Kusumawardani, S. S., & Alfarozi, S. A. (2023). Transformer Encoder Model for Sequential Prediction of Student Performance Based on Their Log Activities. *11*, págs. 18960-18971. IEEE. doi:10.1109/ACCESS.2023.3246122

Kuzilek, J., Hlosta, M., & Zdrahal, Z. (2017). Open University Learning Analytics dataset. *Scientific Data, 4*(170171).

Luo, H., Yu, Q., Xie, S., Wu, H., & Xu, K. (2024). GLU-Transformer for Predicting Withdrawal in VLE. *ICBAR '24: Proceedings of the 2024 4th International Conference on Big Data, Artificial Intelligence and Risk Management*, 835-840.

Papadogiannis, I., Wallace, M., & Karountzou, G. (2024). Educational Data Mining: A Foundational Overview. *Encyclopedia, 4*(4), 1644-1664.

Parlamento Europeo y Consejo de la Unión Europea. (2024). *Reglamento (UE) 2024/1689 por el que se establecen normas armonizadas sobre*

inteligencia artificial (Ley de Inteligencia Artificial). Bruselas. Obtenido de http://data.europa.eu/eli/reg/2024/1689/o

Sejnowski, T. J. (2020). The unreasonable effectiveness of deep learning in artificial intelligence. *Proc Natl Acad Sci U S A.* doi:10.1073/pnas.1907373117

Soepriyanto, Y., Nugroho, R. P., Nahri, M. H., Kesuma, D. W., & Setiasih, M. (2025). From logs to insights: A comprehensive framework for data-driven learning insights. *Jurnal Inovasi Teknologi Pendidikan, 12*(1), 40-49. doi:https://doi.org/10.21831/jitp.v12i1.77432

Torkhani, W., & Rezgui, K. (2025). OULAD MOOC Student Performance. (págs. 228-241). Atlantis Press. doi:10.2991/978-94-6463-654-3_18

Apéndice I: Repositorio de Código y Reproducibilidad

Acceso al Repositorio Todo el código fuente, los notebooks de exploración (EDA) y los scripts de entrenamiento desarrollados para este Trabajo de Fin de Máster se encuentran alojados en un repositorio público de GitHub para garantizar la transparencia y reproducibilidad de los resultados.

Atributo	Detalle
URL del Repositorio	https://github.com/MichaeLaudrup/Artificial_Intelligence_Msc/tree/main/TFM_education_ai_analytics
Plataforma	GitHub
Lenguaje Principal	Python 3.12+
Librerías Clave	Pandas, TensorFlow/Keras, Scikit-learn, SHAP
Licencia	MIT License (Código abierto)
Código QR para acceso directo	

Tabla 1 Ficha técnica de repositorio código fuente

Estructura del Repositorio: El proyecto sigue una arquitectura modular inspirada en los estándares de la industria para ciencia de datos, lo que garantiza la separación de preocupaciones y facilita la reproducibilidad del

experimento. A continuación, se detalla la jerarquía de directorios y la función de sus componentes principales:

root/: Directorio principal del proyecto que contiene los archivos de configuración y automatización.

- data/: Almacén central de datos, organizado por etapas de madurez:
    - raw/: Copias inalteradas de los datos originales (OULAD).
    - interim/: Datos transformados durante los procesos de limpieza intermedios.
    - processed/: Conjuntos de datos finales, anonimizados y listos para alimentar los modelos de IA.
    - external/: Datos complementarios de terceras fuentes utilizados en el análisis.
- notebooks/: Cuadernos interactivos (.ipynb) utilizados para la experimentación inicial.
    - data_exploration.ipynb: Documenta el análisis exploratorio (EDA), visualizaciones estadísticas y la validación de hipótesis iniciales.
- educational_ai_analytics/: Núcleo del código fuente desarrollado como un paquete modular de Python:
    - dataset.py: Funciones para la extracción y carga eficiente de los datos.
    - features.py: Implementación de la ingeniería de variables y transformadores.
    - modeling/: Scripts dedicados al entrenamiento, validación cruzada y serialización de los modelos.
    - plots.py: Motor de visualización personalizado para la generación de gráficas presentadas en este trabajo.
    - config.py: Definición de hiperparámetros, rutas del sistema y constantes globales.

- models/: Directorio destinado a almacenar los pesos de los modelos entrenados y los archivos de metadatos asociados.
- reports/: Resultados del análisis.
    - figures/: Gráficas y representaciones visuales exportadas en alta resolución para la memoria del TFM.
- references/: Documentación técnica, diccionarios de datos y manuales de las fuentes de información utilizadas.
- Makefile: Archivo de automatización que permite replicar todo el flujo de trabajo (limpieza, procesado y entrenamiento) mediante comandos simples (ej. make data, make models).
- pyproject.toml: Especificación de la versión de Python (3.12+) y lista exhaustiva de dependencias necesarias para recrear el entorno virtual.

Apendice II: Tablas de características

Características del autoencoder.

Característica	Descripción
imd_band	Índice de Privación Múltiple (proxy de nivel socioeconómico), mapeado numéricamente.
age_band	Rango de edad del estudiante categorizado numéricamente.
highest_education	Nivel educativo previo del estudiante (escala ordinal).
num_of_prev_attempts	Número de veces que el alumno ha intentado superar el módulo previamente.
studied_credits	Carga académica total (créditos) matriculada simultáneamente.
region_encoded	Región geográfica de residencia del estudiante, codificada numéricamente.
investigated_platform	Variable binaria (0/1) que indica si el alumno accedió a la plataforma antes de iniciar el curso.
prestart_clicks_total	Volumen total de clics registrados antes del inicio oficial.
prestart_active_days	Número total de días distintos en los que hubo actividad previa al inicio.
prestart_active_weeks	Número total de semanas distintas con actividad antes de arrancar.
prestart_earliest_day	Día exacto (en negativo) en el que el alumno realizó su primera conexión histórica.
prestart_anticipation	Valor absoluto del primer día de conexión (días de antelación al inicio).
prestart_intensity	Ratio de intensidad pre-curso (clics totales divididos entre días activos).
clicks_[tipo_actividad]	18 variables independientes que acumulan los clics normalizados por cada tipo de recurso (ej. clicks_forumng, clicks_homepage, clicks_quiz, clicks_oucontent, etc.).
total_weighted_engag	Compromiso acumulado aplicando ponderaciones

ement	(ej. +peso a cuestionarios, -peso a lecturas).
rel_eng_zscore	Proxy del compromiso (total_weighted_engagement), normalizado (Z-score) por curso.
last_week_clicks_weighted	Volumen de compromiso ponderado generado específicamente en la última semana evaluada.
momentum_2w	Aceleración del esfuerzo: compara la actividad de las últimas 2 semanas frente a las 2 previas.
effort_slope	Pendiente de regresión lineal sobre los clics semanales (tendencia general de esfuerzo).
temporal_entropy_uptoW	Entropía de la distribución temporal (valores altos = constancia; bajos = picos esporádicos).
weekly_std_uptoW	Desviación estándar de los clics semanales (mide la volatilidad en el ritmo de estudio).
activity_diversity	Cantidad de tipos de recursos diferentes (foros, wikis, pdfs) que el estudiante ha utilizado.
curiosity_index	Ratio entre la diversidad de recursos utilizados y el número de semanas activas.
top1_share	Proporción de la actividad total concentrada exclusivamente en el recurso más utilizado.
prestart_ratio	Proporción de los clics totales del curso que ocurrieron antes del Día 0.
early_weeks_ratio	Proporción del esfuerzo total concentrado en las primeras 4 semanas del curso.
first_active_week	Semana exacta en la que el estudiante registró su primera interacción post-inicio.
active_weeks	Número total de semanas en las que el alumno interactuó con el VLE.
active_ratio_uptoW	Porcentaje de semanas con actividad respecto al total de semanas transcurridas.
streak_max_uptoW	Racha máxima histórica de semanas consecutivas con actividad.
streak_final	Racha de semanas consecutivas de actividad que llega viva hasta la semana actual (W-1).
weeks_since_last_activity	Semanas de inactividad transcurridas desde el último clic (indicador clave de riesgo).
has_submitted	Variable binaria (0/1) que indica si el alumno ha

	entregado al menos una actividad.
submission_count	Número total de evaluaciones entregadas hasta la fecha de corte.
avg_score	Calificación media obtenida en las tareas entregadas (0-100).
score_std	Desviación estándar de las calificaciones (estabilidad del rendimiento del alumno).
pass_ratio	Porcentaje de entregas que superan la nota mínima de aprobación (40%).
late_ratio	Porcentaje de entregas realizadas fuera de la fecha límite oficial.
score_slope	Tendencia de las calificaciones (pendiente ascendente o descendente en el rendimiento).
weeks_since_last_submission	Semanas transcurridas desde que el alumno realizó su última entrega evaluable.
api_index	Índice compuesto que premia conjuntamente buenas notas y volumen de entregas.

Anexos I: Estructura y diccionario de datos OULAD.

- Tabla I: Dimensión del estudiante (students_processed.csv)

Esta tabla maestra consolida la información estática y demográfica, actuando como el perfil base de cada alumno matriculado.

Campo	Descripción
code_module	Identificador del curso (ej. AAA, BBB).
code_presentation	Semestre en el que se imparte (ej. 2013J, 2014B).
id_student	ID único anonimizado del estudiante.
gender	Género del estudiante (M/F).
region	Región geográfica de residencia en UK.
highest_education	Nivel educativo previo (ej. A Level, HE Qualification).
imd_band	Índice de Privación Múltiple (nivel socioeconómico; % más bajo indica mayor pobreza).
age_band	Rango de edad (0-35, 35-55, 55<=).
num_of_prev_attempts	Número de veces que el estudiante ha intentado este módulo previamente.
studied_credits	Carga total de créditos que el estudiante cursa actualmente.
disability	Indicador de discapacidad declarada (Y/N).
final_result	Variable Objetivo: Pass, Distinction, Fail, Withdrawn.
date_registration	Día relativo al inicio del curso en que se formalizó la matrícula.
date_unregistration	Día relativo en que se produjo la baja (si aplica).

Tabla 2 Descripción de datos de estudiantes

- Tabla II: Dimensión de Rendimiento (assessments_processed.csv)

Contiene los resultados cuantitativos de los hitos evaluativos, permitiendo trazar la evolución académica del alumno.

Campo	Descripción

id_assessment	ID único de la prueba específica.
id_student	ID del estudiante.
date_submitted	Día relativo al inicio del curso en que se realizó la entrega.
is_banked	Indicador de si la nota ha sido convalidada de una presentación anterior.
score	Calificación obtenida (escala 0-100).
assessment_type	Tipología: TMA (Corrección por tutor), CMA (Corrección automática), Exam.
date	Fecha límite oficial de entrega (*Deadline*).
weight	Ponderación de la evaluación en la nota final del curso.

Tabla 3 Descripción de entrega de actividades

Tabla III: Dimensión de Comportamiento (interactions_processed.csv)

Representa el log de actividad diario en el Entorno Virtual de Aprendizaje (VLE). Dada su granularidad, constituye la fuente principal para el modelado secuencial.

Campo	Descripción
code_module	Identificador del curso (ej. AAA, BBB).
code_presentation	Semestre en el que se imparte (ej. 2013J, 2014B).
id_student	ID único anonimizado del estudiante.
gender	Género del estudiante (M/F).
region	Región geográfica de residencia en UK.
highest_education	Nivel educativo previo (ej. A Level, HE Qualification).
imd_band	Índice de Privación Múltiple (nivel socioeconómico; % más bajo indica mayor pobreza).
age_band	Rango de edad (0-35, 35-55, 55<=).
num_of_prev_attempts	Número de veces que el estudiante ha intentado este módulo previamente.
studied_credits	Carga total de créditos que el estudiante cursa actualmente.
disability	Indicador de discapacidad declarada (Y/N).
final_result	Variable Objetivo: Pass, Distinction, Fail, Withdrawn.
date_registration	Día relativo al inicio del curso en que se

	formalizó la matrícula.
date_ unregistration	Día relativo en que se produjo la baja (si aplica).

Tabla 4 Descripción datos de interacción de estudiantes

"Se recomienda que la memoria del TFM tenga un número de páginas ***de contenido*** *comprendido entre 40 y 80 (****en ningún caso inferior a 40****) sin contar dedicatorias, índices, resumen, anexos."*